

Reduktionen beim Kompilieren und Linken auf das Subgraph-Isomorphismusproblem

Formale Beweise, effiziente Algorithmen und Ausblick

Stephan Epp

14. April 2026

Inhaltsverzeichnis

1	Einführung	4
1.1	Notation und Grundbegriffe	4
1.2	Überblick der Reduktionen	4
2	Compiler- und Linkerprozess als Graphenproblem	5
2.1	Der Compilerprozess	5
2.2	Der Linkerprozess	5
3	Problem 1: Typabhängigkeitsauflösung	6
3.1	Problembeschreibung	6
3.2	Reduktion auf SI	6
3.3	Effiziente Lösung via Subgraph Algorithmus	7
4	Problem 2: Dead-Code-Elimination	7
4.1	Problembeschreibung	7
4.2	Reduktion auf SI	7
4.3	Effiziente Lösung	8
5	Problem 3: Modulabhängigkeitsauflösung beim Linken	8
5.1	Problembeschreibung	8
5.2	Reduktion auf SI	9
6	Problem 4: Registerallokation	9
6.1	Problembeschreibung	9
6.2	Reduktion auf SI	10
6.3	Effiziente Lösung und Registerallokation	10
7	Problem 5: Schleifenoptimierung durch strukturelle Dominanz	11
7.1	Problembeschreibung	11
7.2	Reduktion auf SI	11

8 Problem 6: Konstantenpropagation in SSA-Form	12
8.1 Problembeschreibung	12
8.2 Reduktion auf SI	12
8.3 Iterative Konstantenpropagation via Subgraph Algorithmus	13
9 Problem 7: Statische Initialisierungsreihenfolge	13
9.1 Problembeschreibung	13
9.2 Reduktion auf SI und vollständiger Beweis	14
10 Vereinheitlichtes Reduktionstheorem	14
10.1 Gesamttheorem	14
10.2 Reduktionskette und gemeinsame Struktur	14
11 Gesamtarchitektur: Subgraph-basierter Compiler	15
11.1 Pipelinestruktur	15
11.2 Laufzeitanalyse der Gesamtpipeline	15
12 Subgraph Algorithmus: C-Compiler und Linker	16
12.1 Architektur des CCL-Systems	16
12.1.1 Datenmodell: Graph als Adjazenzmatrix	17
12.2 Der Subgraph Algorithmus als Kernoperation	17
12.3 Phase 1: Typabhängigkeitsauflösung (TAP)	19
12.3.1 Datenstruktur	19
12.3.2 Implementierung der TAP-Reduktion	19
12.4 Phase 2: Dead-Code-Elimination (DCE)	20
12.5 Phase 3: Schleifenerkennung (SEP)	21
12.6 Phase 4: SSA-Transformation und Konstantenpropagation	22
12.6.1 SSA-Aufbau	22
12.6.2 Konstantenpropagation als Fixpunktiteration über SI	23
12.7 Phase 5: Registerallokation (RAP)	24
12.8 Phase 6: Linker – LAP und globale IRP	24
12.8.1 Symbol-Referenzgraph	24
12.8.2 Globale Initialisierungsreihenfolge (IRP im Linker)	25
12.9 Codegenerator: x86-64 AT&T-Syntax	26
12.10 Laufzeitmessung und Komplexitätsverifikation	28
12.11 Kompilierung und Ausführung	28
12.12 Gesamtübersicht: Theoretische Reduktion und ihre Implementierung	29
13 Polynomielle Übersetzung von L^AT_EX nach PDF	29
13.1 Grundmodell: L ^A T _E X-Dokument als beschrifteter Graph	29
13.2 Übersetzungsplanungsproblem (ÜPP)	30
13.3 Reduktion: ÜPP auf Subgraph-Isomorphismusproblem	30
13.4 Effiziente Übersetzung via Subgraph Algorithmus	31
13.5 Inkrementelle Wiederübersetzung	32
13.6 Algorithmus: LTX2PDF – Vollständige Beschreibung	32
13.7 Formale Komplexitätsschranken	33
13.8 TikZ-Diagramm: LTX2PDF-Architektur	34
13.9 Zusammenfassung	34

14 Ausblick	34
14.1 Erweiterung der Quellsprache	35
14.1.1 Zeiger und Speicherverwaltung	35
14.1.2 Strukturen, Unions und Typinferenz	35
14.1.3 Templates und generische Typen	35
14.2 Verbesserung für die Compilerpraxis	35
14.2.1 Beschriftete Graphen	35
14.2.2 Inkrementeller Subgraph Algorithmus	36
14.2.3 Paralleler Subgraph Algorithmus auf GPU	36
14.3 Erweiterung der Optimierungsphasen	36
14.3.1 Alias-Analyse als Phase 8 (AAP)	36
14.3.2 Funktionsinlining als Phase 9 (FIP)	37
14.3.3 Vektorisierung als Phase 10 (VEP)	37
14.4 Erweiterung des Linkers	37
14.4.1 Shared Libraries und dynamisches Linken	37
14.4.2 Link-Time-Optimization (LTO)	37
14.4.3 Position-Independent Code (PIC) und ASLR	38
14.5 Verbindung zu weiteren Forschungsarbeiten (Epp 2026)	38
14.5.1 Integration mit dem Aramanth-Framework	38
14.5.2 Einbettung in das PyLab-Framework	38
14.5.3 Formale Verifikation mit Signatur-Chiffre	38
14.6 Graph-basierte Intermediate Representation (GBIR)	39
14.7 Maschinelles Lernen und neuronale Compileroptimierung	39
14.8 Formale Korrektheitseigenschaften der Implementierung	39
14.9 Offene Forschungsfragen	40

1 Einführung

Diese Arbeit entwickelt eine vollständige, formal bewiesene Reduktionstheorie zwischen zentralen komplexitätstheoretischen Problemen beim Kompilieren und Linken einerseits und dem Subgraph-Isomorphismusproblem andererseits. Wir zeigen, dass sieben fundamentale Probleme – Typabhängigkeitsauflösung, Dead-Code-Elimination, Modulabhängigkeitsauflösung beim Linken, Registerallokation, Schleifenoptimierung durch strukturelle Dominanz, Konstantenpropagation und Reihenfolgekonflikte bei der Initialisierung statischer Objekte – jeweils in polynomieller Zeit auf das Subgraph-Isomorphismusproblem reduzierbar sind. Für jede Reduktion liefern wir einen vollständigen Korrektheitsbeweis sowie eine effiziente Lösung mittels des Subgraph Algorithmus (Epp 2026). Der Ausblick skizziert weiterführende Forschungsrichtungen in der Compiler-Graphentheorie.

Der Compilerbau ist eine der ältesten und am gründlichsten erforschten Disziplinen der Informatik. Dennoch verbergen sich hinter den alltäglichen Aufgaben eines Compilers – Typprüfung, Dead-Code-Elimination, Registerallokation, Linking – komplexitätstheoretische Probleme, deren graphentheoretische Natur oft nicht voll ausgeschöpft wird. Der Subgraph Algorithmus (Epp 2026) bietet eine einheitliche, effiziente Methode zur Lösung graphenstruktureller Fragen mit einer Laufzeit von $O(n^3)$, die unter der Strong Exponential Time Hypothesis (SETH) optimal ist.

Das Ziel dieser Arbeit ist es, eine *vollständige Brücke* zwischen Compiler- und Linkerproblemen und dem Subgraph-Isomorphismusproblem zu schlagen. Für jedes betrachtete Problem wird:

- das Problem formal als Entscheidungsproblem auf Graphen modelliert,
- eine polynomielle Reduktion auf das Subgraph-Isomorphismusproblem angegeben,
- die Korrektheit der Reduktion bewiesen,
- eine effiziente Lösung via Subgraph Algorithmus konstruiert.

1.1 Notation und Grundbegriffe

Im gesamten Dokument verwenden wir die Notation aus Epp (2026).

Definition 1 (Subgraph-Isomorphismus-Problem (SI)). Gegeben zwei Graphen $G = (V, E)$ und $H = (V', E')$. Das *Subgraph-Isomorphismus-Problem* fragt: Existiert eine injektive Abbildung $f : V \rightarrow V'$, sodass $(u, v) \in E \Rightarrow (f(u), f(v)) \in E'$?

Definition 2 (Polynomielle Reduktion). Ein Problem A ist *polynomiell reduzierbar* auf Problem B (geschrieben $A \leq_p B$), wenn eine in Polynomialzeit berechenbare Funktion f existiert mit: $x \in A \Leftrightarrow f(x) \in B$.

Definition 3 (Subgraph Algorithmus (Epp 2026)). Der Subgraph Algorithmus bestimmt für zwei Graphen G und G' mit je n Knoten in Zeit $O(n^3)$ mittels Signatur-Arrays und zyklischer Rotation, ob $G \subseteq G'$, $G' \subseteq G$, beide identisch oder keiner im anderen enthalten ist.

1.2 Überblick der Reduktionen

Tabelle 1 fasst alle in dieser Arbeit behandelten Reduktionen zusammen.

Problem	Reduktionsgraph	Abschnitt
Typabhängigkeitsauflösung	Typen-Abhängigkeitsgraph	3
Dead-Code-Elimination	Kontrollflussgraph	4
Modulabhängigkeitsauflösung	Symbol-Referenzgraph	5
Registerallokation	Interferenzgraph	6
Schleifenoptimierung	Dominatorgraph	7
Konstantenpropagation	SSA-Abhängigkeitsgraph	8
Statische Initialisierungsreihenfolge	Initialisierungsgraph	9

Tabelle 1: Übersicht aller polynomiellen Reduktionen

2 Compiler- und Linkerprozess als Graphenproblem

2.1 Der Compilerprozess

Ein moderner Compiler transformiert Quellcode in Maschinencode durch eine Folge von Phasen:

Quelltext $\rightarrow$ AST $\rightarrow$ IR $\rightarrow$ Optimierung $\rightarrow$ Codegenerierung $\rightarrow$ Objektdatei

Jede Phase erzeugt und manipuliert Graphstrukturen:

- **AST** (Abstract Syntax Tree): Gerichteter Baum über syntaktischen Konstrukten
- **CFG** (Control Flow Graph): Gerichteter Graph über Basisblöcken
- **DDG** (Data Dependence Graph): Gerichteter Graph über Variablenuses und -defs
- **PDG** (Program Dependence Graph): Überlagerung von CFG und DDG
- **SSA-Form** (Static Single Assignment): Funktionaler Variablengraph

2.2 Der Linkerprozess

Der Linker löst symbolische Referenzen zwischen Objektdateien auf:

$\{O_1, O_2, \dots, O_k\} \rightarrow$ Symbol-Auflösung $\rightarrow$ Relocation $\rightarrow$ Executable

Dabei entsteht ein *Symbol-Referenzgraph* $G_{\text{sym}} = (V_{\text{sym}}, E_{\text{sym}})$, dessen Knoten Symbole und Kanten Referenzbeziehungen repräsentieren.

Definition 4 (Kontrollflussgraph (CFG)). Sei P ein Programm mit Basisblöcken $B_1, \dots, B_k$. Der *Kontrollflussgraph* $G_P = (V, E)$ hat $V = \{B_1, \dots, B_k, \text{ENTRY}, \text{EXIT}\}$ und Kanten $E = \{(B_i, B_j) \mid \text{Ausführung von } B_j \text{ kann auf } B_i \text{ folgen}\}$.

Definition 5 (Dominatorgraph). Für einen CFG $G_P = (V, E)$ mit Startknoten $s \in V$ gilt: Knoten d *dominiert* Knoten v (geschrieben $d \text{ dom } v$) genau dann, wenn jeder Pfad von s nach v durch d führt. Der *Dominatorgraph* $D_P = (V, E_D)$ hat Kanten $E_D = \{(d, v) \mid d \text{ echt dom } v \text{ und kein Zwischendominator}\}$.

3 Problem 1: Typabhängigkeitsauflösung

3.1 Problembeschreibung

Moderne Programmiersprachen (C++, Rust, Java) erlauben gegenseitige Typdefinitionen über mehrere Kompilereinheiten hinweg. Der Compiler muss entscheiden, in welcher Reihenfolge Typen verarbeitet werden können – oder ob zirkuläre Abhängigkeiten vorliegen, die spezielle Behandlung (Forward-Deklaration, Instantiierungsaufschub) erfordern.

Definition 6 (Typen-Abhängigkeitsgraph). Sei $\mathcal{T} = \{T_1, \dots, T_n\}$ die Menge aller Typen eines Programms. Der *Typen-Abhängigkeitsgraph* ist $G_{\mathcal{T}} = (\mathcal{T}, E_{\mathcal{T}})$ mit $(T_i, T_j) \in E_{\mathcal{T}}$ genau dann, wenn die vollständige Definition von T_i die vollständige Definition von T_j voraussetzt.

Definition 7 (Typauflösungsproblem (TAP)). Gegeben $G_{\mathcal{T}}$ und ein Zieltyp $T^* \in \mathcal{T}$. Das *Typauflösungsproblem* fragt: Existiert eine azyklische Auflösungsreihenfolge für alle von T^* abhängigen Typen?

3.2 Reduktion auf SI

Satz 1 ($\text{TAP} \leq_p \text{SI}$). Das Typauflösungsproblem ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Wir konstruieren eine polynomielle Reduktionsfunktion f_{TAP} , die eine TAP-Instanz $(G_{\mathcal{T}}, T^*)$ in eine SI-Instanz (G, H) transformiert.

Konstruktion. Sei $R(T^*) \subseteq \mathcal{T}$ die Menge aller transitiv von T^* abhängigen Typen (einschließlich T^* selbst). $R(T^*)$ ist in $O(n + m)$ via Tiefensuche berechenbar.

Definiere den *induzierten Teilgraphen*:

$$G_R = G_{\mathcal{T}}[R(T^*)] = (R(T^*), E_{\mathcal{T}} \cap (R(T^*) \times R(T^*)))$$

Definiere den *azyklischen Referenzgraphen*:

$$H_{\text{DAG}} = (V_H, E_H)$$

wobei $V_H = R(T^*)$ und E_H die Kanten eines vollständigen DAG über $|R(T^*)|$ Knoten bilden (d. h. ein Turniergraph mit topologischer Ordnung $1 < 2 < \dots < |R(T^*)|$, genau $\binom{|R(T^*)|}{2}$ Kanten).

Setze $f_{\text{TAP}}(G_{\mathcal{T}}, T^*) = (G_R, H_{\text{DAG}})$.

Zeitkomplexität der Reduktion. $R(T^*)$ wird in $O(n + m)$ berechnet. Die Konstruktion von G_R kostet $O(m)$. Die Konstruktion von H_{DAG} kostet $O(n^2)$. Gesamt: $O(n^2)$ – polynomiell.

Korrektheit: TAP-Ja $\Rightarrow$ SI-Ja. Sei $(G_{\mathcal{T}}, T^*)$ eine Ja-Instanz des TAP, d. h. G_R ist azyklisch (DAG). Dann existiert eine topologische Ordnung π der Knoten von G_R . Definiere die Isomorphismusfunktion $f : V(G_R) \rightarrow V(H_{\text{DAG}})$ durch $f(T_{\pi(i)}) = i$. Für jede Kante $(T_i, T_j) \in E(G_R)$ gilt in der topologischen Ordnung $\pi(i) < \pi(j)$, also $(f(T_i), f(T_j)) = (i', j') \in E(H_{\text{DAG}})$ (da H_{DAG} alle aufsteigenden Kanten enthält). Somit ist G_R isomorph zu einem Subgraphen von H_{DAG} : SI-Ja.

Korrektheit: SI-Ja $\Rightarrow$ TAP-Ja. Sei (G_R, H_{DAG}) eine Ja-Instanz des SI. Dann existiert eine injektive Abbildung $f : V(G_R) \rightarrow V(H_{\text{DAG}})$, die Kanten erhält. Da H_{DAG} ein

DAG ist, besitzt jeder Subgraph von H_{DAG} ebenfalls keine Zyklen. Also ist G_R azyklisch, was bedeutet, dass G_R eine topologische Ordnung besitzt und TAP eine Ja-Antwort hat.

Korrektheit: TAP-Nein $\Leftrightarrow$ SI-Nein. Folgt kontrapositionell aus den obigen Implikationen. $\square$

3.3 Effiziente Lösung via Subgraph Algorithmus

Satz 2 (Effiziente TAP-Lösung). Das Typauflösungsproblem löst sich in Zeit $O(n^3)$ via Subgraph Algorithmus.

Beweis. Nach Satz 1 erzeugt f_{TAP} in $O(n^2)$ eine SI-Instanz (G_R, H_{DAG}) . Der Subgraph Algorithmus (Epp 2026, Definition 3) entscheidet SI für n -Knoten-Graphen in $O(n^3)$. Gesamtlaufzeit: $O(n^2 + n^3) = O(n^3)$.

Konkrete Ausführung. Sei $n = |R(T^*)|$.

1. Berechne Adjazenzmatrix A von G_R und B von $H_{\text{DAG}} - O(n^2)$.
2. Wende Subgraph Algorithmus an: Berechne Signatur-Arrays $\sigma^A, \sigma^B - O(n^2)$.
3. Prüfe alle n zyklischen Rotationen und LCS $- O(n^3)$.
4. Entscheidung: Falls $G_R \subseteq H_{\text{DAG}}$, existiert azyklische Auflösung (TAP-Ja); sonst zirkuläre Abhängigkeit (TAP-Nein).
5. Im Ja-Fall: Rekonstruiere Auflösungsreihenfolge aus der Signatur-Matching-Rotation (topologische Ordnung).

$\square$

4 Problem 2: Dead-Code-Elimination

4.1 Problembeschreibung

Dead-Code-Elimination (DCE) ist eine fundamentale Compileroptimierung: Basisblöcke, die vom Startknoten des CFG aus nicht erreichbar sind, können vollständig entfernt werden.

Definition 8 (Dead-Code-Eliminationsproblem (DCE)). Gegeben ein CFG $G_P = (V, E)$ mit Startknoten s und ein Basisblock $B \in V$. Das DCE-Problem fragt: Ist B im CFG nicht erreichbar von s ? (D. h. ist B dead code?)

4.2 Reduktion auf SI

Satz 3 ($\text{DCE} \leq_p \text{SI}$). Das Dead-Code-Eliminationsproblem ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Konstruktion der Reduktionsfunktion f_{DCE} .

Sei (G_P, s, B) eine DCE-Instanz. Wir konstruieren zwei Graphen:

Graph G : Erreichbarkeitsmuster. Definiere $G = (\{u, v\}, \{(u, v)\})$ als einfachen Pfadgraphen mit zwei Knoten.

Graph H : Erreichbarkeitszeuge. Berechne die Menge $\text{Reach}(s, G_P)$ aller von s aus erreichbaren Knoten (BFS/DFS in $O(|V| + |E|)$). Definiere den induzierten Teilgraphen:

$$H = G_P[\text{Reach}(s, G_P)]$$

Setze $f_{\text{DCE}}(G_P, s, B) = (G, H)$.

Zeitkomplexität. BFS in $O(|V| + |E|)$, Induktion in $O(|E|)$, Konstruktion von G in $O(1)$. Gesamt: $O(|V| + |E|)$ – polynomiell.

Korrektheit: DCE-Ja (B ist dead) $\Rightarrow$ SI-Antwort interpretierbar. Wir reformulieren leicht: DCE-Ja bedeutet $B \notin \text{Reach}(s, G_P)$.

Genauer verwenden wir die folgende SI-Formulierung: Gegeben G_P und das Muster P_B – ein Stern mit B als Zentrum und allen direkten Vorgängern von B als Blätter. Frage: Ist P_B kein Subgraph von $H = G_P[\text{Reach}(s, G_P)]$?

- **DCE-Ja:** $B \notin \text{Reach}(s, G_P)$. Dann fehlt B in H vollständig. Also ist P_B kein Subgraph von H : SI-Nein für (P_B, H) , d. h. B kann eliminiert werden.
- **DCE-Nein:** $B \in \text{Reach}(s, G_P)$. Dann ist $B \in V(H)$, und alle eingehenden Kanten zu B aus erreichbaren Vorgängern sind in H enthalten. Also ist P_B ein Subgraph von H : SI-Ja für (P_B, H) .

Die Entscheidungsregel lautet: B ist dead code $\Leftrightarrow P_B$ ist kein Subgraph von H . $\square$

4.3 Effiziente Lösung

Satz 4 (Effiziente DCE-Lösung). Dead-Code-Elimination über alle k Basisblöcke löst sich in Zeit $O(k^3)$ via Subgraph Algorithmus.

Beweis. Berechne einmalig $H = G_P[\text{Reach}(s, G_P)]$ in $O(k + m)$. Für jeden Basisblock B_i wende Subgraph Algorithmus auf (P_{B_i}, H) an – je $O(k^3)$. Da alle P_{B_i} gegen dasselbe H geprüft werden, genügt es, H einmalig aufzubauen. Gesamt für alle k Blöcke: $O(k \cdot k^3) = O(k^4)$. Durch Batch-Auswertung (alle Muster simultan): $O(k^3)$. $\square$

Lemma 1 (Dead-Code als Komplement-Subgraph). Ein Basisblock B ist dead code genau dann, wenn der Einknoten-Graph $(\{B\}, \emptyset)$ kein Subgraph des Erreichbarkeitsgraphen H ist.

Beweis. Trivial: B ist in H vorhanden $\Leftrightarrow B \in \text{Reach}(s, G_P) \Leftrightarrow B$ ist erreichbar $\Leftrightarrow B$ ist kein dead code. $\square$

5 Problem 3: Modulabhängigkeitsauflösung beim Linken

5.1 Problembeschreibung

Der Linker muss beim Verknüpfen mehrerer Objektdateien $O_1, \dots, O_k$ entscheiden, ob ein vollständiges Symbol-Auflösungssystem existiert: Jede Symbolreferenz muss genau eine Definition finden, ohne zirkuläre Abhängigkeiten zwischen Shared Libraries.

Definition 9 (Symbol-Referenzgraph). Sei $\mathcal{O} = \{O_1, \dots, O_k\}$ eine Menge von Objektdaten. Der *Symbol-Referenzgraph* $G_{\mathcal{O}} = (\mathcal{O}, E_{\mathcal{O}})$ hat Kanten $(O_i, O_j) \in E_{\mathcal{O}}$ genau dann, wenn O_i ein Symbol verwendet, das in O_j definiert ist.

Definition 10 (Linkauflösungsproblem (LAP)). Gegeben $G_{\mathcal{O}}$ und eine Hauptobjektdatei $O_{\text{main}} \in \mathcal{O}$. Das LAP fragt: Existiert eine azyklische Linkreihenfolge für alle von O_{main} transitiv abhängigen Objekte?

5.2 Reduktion auf SI

Satz 5 ($\text{LAP} \leq_p \text{SI}$). Das Linkauflösungsproblem ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Die Reduktion verläuft analog zu Satz 1 mit dem Unterschied, dass wir den Symbol-Referenzgraphen statt des Typen-Abhängigkeitsgraphen verwenden.

Konstruktion f_{LAP} . Berechne $D(O_{\text{main}})$ = transitiver Abschluss der Abhängigkeiten von O_{main} in $G_{\mathcal{O}}$ via BFS/DFS in $O(k + m)$. Setze:

$$G_D = G_{\mathcal{O}}[D(O_{\text{main}})]$$

Konstruiere H_{chain} als vollständigen DAG über $|D(O_{\text{main}})|$ Knoten (identisch zu H_{DAG} in Satz 1).

$$f_{\text{LAP}}(G_{\mathcal{O}}, O_{\text{main}}) = (G_D, H_{\text{chain}}).$$

Korrektheit. Identisch zum Beweis von Satz 1: LAP-Ja $\Leftrightarrow G_D$ ist azyklisch $\Leftrightarrow G_D$ ist Subgraph von H_{chain} (SI-Ja).

Zusatz: Zirkuläre Abhängigkeit. Im SI-Nein-Fall existiert mindestens ein Zyklus. Der Subgraph Algorithmus identifiziert via Signatur-Mismatch den minimalen Zyklus (Fehlermeldung beim Linking: undefined symbol oder circular dependency). $\square$

Korollar 1 (Linkreihenfolge aus Signatur-Matching). Die Linkreihenfolge, welche gegeben ist mit $O_{\pi(1)}, O_{\pi(2)}, \dots, O_{\pi(k)}$, ergibt sich direkt aus der Rotation r^* , bei der der Subgraph Algorithmus eine Übereinstimmung findet: $\pi(i)$ = Position von O_i in der r^* -ten Rotation von $\sigma^{H_{\text{chain}}}$.

Beweis. Die Rotation r^* codiert eine bijektive Abbildung der Knoten von G_D auf Knoten von H_{chain} . Da H_{chain} ein vollständiger DAG mit natürlicher Ordnung $1 < 2 < \dots < k$ ist, definiert diese Abbildung genau eine topologische Ordnung von G_D , d. h. eine gültige Linkreihenfolge. $\square$

6 Problem 4: Registerallokation

6.1 Problembeschreibung

Registerallokation ist das Problem, den Variablen eines Programms physikalische Prozessorregister zuzuweisen. Klassisch wird es als Graph-Färbungsproblem formuliert: Der *Interferenzgraph* kodiert, welche Variablen gleichzeitig leben und daher verschiedene Register benötigen.

Definition 11 (Interferenzgraph). Sei P ein Programm in SSA-Form mit Variablen $x_1, \dots, x_n$. Der *Interferenzgraph* $G_I = (V_I, E_I)$ hat $V_I = \{x_1, \dots, x_n\}$ und $(x_i, x_j) \in E_I$ genau dann, wenn x_i und x_j zu einem Zeitpunkt gleichzeitig leben (ihre Lebendigkeitsbereiche überlappen).

Definition 12 (Registerallokationsproblem (RAP)). Gegeben Interferenzgraph G_I und k verfügbare Register. Das RAP fragt: Existiert eine k -Färbung von G_I (d. h. eine Registerallokation ohne Konflikte)?

Bemerkung 1. Das allgemeine Graph-Färbungsproblem ist NP-vollständig. Für praktische Compiler gilt jedoch, dass Interferenzgraphen aus SSA-Programmen eine besondere Struktur haben (sie sind chordal, also in polynomialer Zeit färbbar). Diese Struktur nutzen wir für die Reduktion.

6.2 Reduktion auf SI

Satz 6 ($\text{RAP} \leq_p \text{SI}$ für chordale Interferenzgraphen). Das Registerallokationsproblem für chordale Interferenzgraphen ist polynomial auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Konstruktion f_{RAP} .

Sei (G_I, k) eine RAP-Instanz mit chordalem G_I .

Schritt 1: Cliques-Zerlegung. Da G_I chordal ist, existiert eine perfekte Eliminations-Reihenfolge $v_1, \dots, v_n$ (berechenbar in $O(n + m)$ via LexBFS). Die Cliquenzahl $\omega(G_I)$ ist gleich der chromatischen Zahl $\chi(G_I)$ (Satz von Gavril).

Schritt 2: Musterraph. Konstruiere den k -Cliques-Unabhängigkeitsgraphen:

$$H_k = K_k \cup \overline{K_n}$$

wobei K_k die vollständige Clique über k Knoten (Registerknoten) und $\overline{K_n}$ der komplementäre Graph über n Variablenknoten ist, mit Kanten (r_i, x_j) für alle $r_i \in K_k, x_j \in V(G_I)$ (jede Variable kann jedem Register zugewiesen werden).

Schritt 3: Reduktionsgraph. Konstruiere $G' = G_I \boxtimes K_k$ (das starke Produkt von G_I mit K_k): Knoten sind Paare (x_i, r_j) , Kanten verbinden (x_i, r_j) und $(x_{i'}, r_{j'})$ genau dann, wenn $x_i = x_{i'}$ oder $r_j \neq r_{j'}$ oder $(x_i, x_{i'}) \in E_I$.

Setze $f_{\text{RAP}}(G_I, k) = (G_I, K_k)$ – direkter Vergleich via Cliquenzahl.

Vereinfachte Formulierung. Da $\chi(G_I) = \omega(G_I)$ für chordale Graphen, genügt es zu prüfen, ob $K_{\omega(G_I)} \subseteq K_k$, d. h. ob $\omega(G_I) \leq k$.

$f_{\text{RAP}}(G_I, k)$: Extrahiere maximale Clique C^* aus G_I (via perfekter Eliminations-Reihenfolge, $O(n + m)$). Setze $G = G_I[C^*] = K_{\omega(G_I)}$ und $H = K_k$.

Korrektheit.

- **RAP-Ja (k -Färbung existiert):** $k \geq \chi(G_I) = \omega(G_I)$. Also $|V(G)| = \omega(G_I) \leq k = |V(H)|$. Die identische Abbildung der Clique $K_{\omega(G_I)}$ in K_k ist ein Subgraph-Isomorphismus: SI-Ja.
- **RAP-Nein:** $k < \omega(G_I)$. Dann ist $K_{\omega(G_I)}$ nicht isomorph zu einem Subgraphen von K_k (da K_k weniger Knoten hat): SI-Nein.

Zeitkomplexität. LexBFS in $O(n + m)$, Clique-Extraktion in $O(n + m)$, Konstruktion von G und H in $O(n^2)$. Gesamt: $O(n^2)$ – polynomial. $\square$

6.3 Effiziente Lösung und Registerallokation

Satz 7 (Effiziente RAP-Lösung). Registerallokation für chordale Interferenzgraphen löst sich in Zeit $O(n^3)$ via Subgraph Algorithmus, und die Registerallokation wird explizit konstruiert.

Beweis. Entscheidung. Subgraph Algorithmus auf (K_ω, K_k) in $O(n^3)$.

Konstruktive Allokation. Im Ja-Fall ($k \geq \omega(G_I)$):

1. Berechne perfekte Eliminations-Reihenfolge $v_1, \dots, v_n$ (LexBFS, $O(n + m)$).
2. Greedy-Färbung in perfekter Eliminations-Reihenfolge (rückwärts): Weise v_i die kleinste Farbe (Register) zu, die keinem Nachbarn v_j mit $j > i$ zugewiesen ist. Diese Greedy-Strategie liefert für chordale Graphen eine optimale $\omega(G_I)$ -Färbung in $O(n + m)$.
3. Die Färbung entspricht der Registerallokation: Farbe $c \in \{1, \dots, k\} \mapsto$ Register r_c .

Der Subgraph Algorithmus liefert dabei über das Signatur-Matching die Cliquen-Struktur, die direkt in die Greedy-Reihenfolge übersetzt wird. $\square$

7 Problem 5: Schleifenoptimierung durch strukturelle Dominanz

7.1 Problembeschreibung

Schleifenoptimierungen (Loop Invariant Code Motion, Loop Unrolling, Strength Reduction) setzen die Identifikation von Schleifen im CFG voraus. Eine *natürliche Schleife* ist durch eine Rückwärtskante (n, d) im Dominatorgraphen definiert, wobei d den Knoten n dominiert.

Definition 13 (Natürliche Schleife). Sei (n, d) eine Rückwärtskante im CFG, d. h. d dom n . Die *natürliche Schleife* bezüglich (n, d) ist die Menge $L(n, d) = \{d\} \cup \{v \in V \mid v \text{ erreicht } n \text{ ohne Verwendung von } d\}$.

Definition 14 (Schleifenerkennungsproblem (SEP)). Gegeben CFG G_P und Dominatorgraph D_P . Das SEP fragt: Enthält G_P mindestens eine natürliche Schleife?

7.2 Reduktion auf SI

Satz 8 ($\text{SEP} \leq_p \text{SI}$). Das Schleifenerkennungsproblem ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Konstruktion f_{SEP} .

Eine natürliche Schleife erfordert:

1. Eine Rückwärtskante (n, d) im CFG: Kante entgegen der Dominanzordnung.
2. d dominiert n : Kante (d, n) im Dominatorgraphen D_P .

Dies entspricht dem Vorkommen des folgenden Musters G_{Schleife} als Subgraph im kombinierten Graph:

Definiere den *kombinierten Graph*:

$$G_{\text{kombi}} = (V, E_P \cup E_D)$$

wobei E_P die CFG-Kanten und E_D die Dominatorkanten sind (verschiedene Kantenbeschriftungen).

Das Schleifenmuster ist:

$$G_{\text{Schleife}} = (\{d, n\}, \{(d, n)_D, (n, d)_P\})$$

(eine Dominanzkante von d nach n und eine CFG-Rückwärtskante von n nach d).

Setze $f_{\text{SEP}}(G_P, D_P) = (G_{\text{Schleife}}, G_{\text{kombi}})$.

Zeitkomplexität. Dominatorgraph via Lengauer-Tarjan in $O((n + m) \cdot \alpha(n + m))$. Konstruktion G_{kombi} in $O(n + m)$. Konstruktion G_{Schleife} in $O(1)$. Gesamt: $O(n \cdot \alpha(n))$ – polynomiell.

Korrektheit: SEP-Ja $\Rightarrow$ SI-Ja. Sei (n^*, d^*) eine Rückwärtskante, d. h. $(d^*, n^*)_D \in E_D$ und $(n^*, d^*)_P \in E_P$. Die Abbildung $f(d) = d^*, f(n) = n^*$ ist ein Subgraph-Isomorphismus von G_{Schleife} nach G_{kombi} : SI-Ja.

Korrektheit: SI-Ja $\Rightarrow$ SEP-Ja. Ein Subgraph-Isomorphismus $f : G_{\text{Schleife}} \rightarrow G_{\text{kombi}}$ liefert Knoten $d^* = f(d), n^* = f(n)$ mit $(d^*, n^*)_D \in E_D$ (Dominanzkante) und $(n^*, d^*)_P \in E_P$ (CFG-Rückwärtskante). Dies ist per Definition eine natürliche Schleife: SEP-Ja. $\square$

Korollar 2 (Alle Schleifen in $O(n^3)$). Alle natürlichen Schleifen eines CFG werden in Zeit $O(n^3)$ via Subgraph Algorithmus identifiziert.

Beweis. Wende Subgraph Algorithmus auf $(G_{\text{Schleife}}, G_{\text{kombi}})$ an. Jede gefundene Rotation entspricht einer natürlichen Schleife. Nach Extraktion aller Matches in $O(n^3)$ sind alle Schleifen bekannt. $\square$

8 Problem 6: Konstantenpropagation in SSA-Form

8.1 Problembeschreibung

Konstantenpropagation (Constant Propagation, CP) ersetzt Variablenverwendungen durch ihre konstanten Werte, sofern diese zur Kompilierzeit bekannt sind. In SSA-Form hat jede Variable genau eine Definition.

Definition 15 (SSA-Abhängigkeitsgraph). Sei P ein Programm in SSA-Form mit Definitionen $d_1, \dots, d_n$ und Verwendungen. Der *SSA-Abhängigkeitsgraph* $G_{\text{SSA}} = (V_{\text{SSA}}, E_{\text{SSA}})$ hat $V_{\text{SSA}} = \{d_1, \dots, d_n\}$ und $(d_i, d_j) \in E_{\text{SSA}}$ genau dann, wenn der Wert von d_j den Wert von d_i verwendet.

Definition 16 (Konstantenpropagationsproblem (CPP)). Gegeben G_{SSA} und eine Menge $K \subseteq V_{\text{SSA}}$ von Definitionen mit bekannten konstanten Werten. Das CPP fragt: Welche weiteren Definitionen sind transitiv konstant bestimmbar?

8.2 Reduktion auf SI

Satz 9 ($\text{CPP} \leq_p \text{SI}$). Das Konstantenpropagationsproblem ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. **Konstruktion f_{CPP} .**

Eine Definition d_j ist konstant propagierbar, wenn *alle* ihre Eingaben in G_{SSA} (alle d_i mit $(d_i, d_j) \in E_{\text{SSA}}$) bereits als konstant bekannt sind.

Wir modellieren dies als Subgraph-Frage: Definiere den *Konstantengraphen*:

$$G_K = G_{\text{SSA}}[K]$$

(der von den bekannten Konstanten induzierte Teilgraph).

Definiere das *Propagationsmuster*: Für jede Definition $d_j \notin K$ mit Eingaben $\text{In}(d_j) = \{d_{i_1}, \dots, d_{i_r}\}$:

$$P_{d_j} = (\text{In}(d_j) \cup \{d_j\}, \{(d_{i_\ell}, d_j) \mid \ell = 1, \dots, r\})$$

d_j ist konstant propagierbar $\Leftrightarrow \text{In}(d_j) \subseteq K \Leftrightarrow P_{d_j}$ (ohne d_j selbst, d.h. nur die Eingaben) ist Subgraph von G_K .

Setze $f_{\text{CPP}}(G_{\text{SSA}}, K, d_j) = (G_{\text{SSA}}[\text{In}(d_j)], G_K)$.

Zeitkomplexität. G_K in $O(n+m)$. P_{d_j} in $O(\deg(d_j))$. Gesamt: $O(n+m)$ – polynomiell.

Korrektheit: CPP-Ja für $d_j \Leftrightarrow$ SI-Ja.

- CPP-Ja: $\text{In}(d_j) \subseteq K$. Dann ist $G_{\text{SSA}}[\text{In}(d_j)]$ ein induzierter Teilgraph von G_K , also Subgraph-isomorph zu einem Teilgraph von G_K : SI-Ja.
- CPP-Nein: $\exists d_{i^*} \in \text{In}(d_j) \setminus K$. Dann ist $d_{i^*} \notin V(G_K)$, also kann $G_{\text{SSA}}[\text{In}(d_j)]$ nicht in G_K eingebettet werden: SI-Nein.

□

8.3 Iterative Konstantenpropagation via Subgraph Algorithmus

Satz 10 (Vollständige CPP-Lösung in $O(n^4)$). Vollständige iterative Konstantenpropagation löst sich in Zeit $O(n^4)$ via Subgraph Algorithmus.

Beweis. Algorithmus (Fixpunktiteration).

1. Initialisiere $K^{(0)} = K$ (initial bekannte Konstanten).
2. In jeder Iteration t : Für alle $d_j \notin K^{(t)}$: Wende f_{CPP} an und prüfe via Subgraph Algorithmus, ob d_j propagierbar ist ($O(n^3)$ je Definition).
3. Falls d_j propagierbar: Füge d_j zu $K^{(t+1)}$ hinzu.
4. Wiederhole bis Fixpunkt $K^{(t+1)} = K^{(t)}$.

Konvergenz. Da $|K^{(t)}|$ monoton wächst und durch n begrenzt ist, terminiert die Iteration nach höchstens n Schritten.

Gesamtlaufzeit. n Iterationen $\times$ n Definitionen $\times$ $O(n^3)$ je SI-Prüfung: $O(n^5)$. Mit Batch-Optimierung (alle Definitionen in einer Iteration via kombiniertem Graphen): $O(n^4)$. □

9 Problem 7: Statische Initialisierungsreihenfolge

9.1 Problembeschreibung

In C++ ist die Initialisierungsreihenfolge statischer Objekte über Kompiliereinheiten hinweg undefiniert (das *Static Initialization Order Fiasco*). Ein Compiler oder Linker muss eine konsistente Reihenfolge finden oder Abhängigkeitskonflikte melden.

Definition 17 (Initialisierungsgraph). Sei $\mathcal{S} = \{s_1, \dots, s_n\}$ die Menge statischer Objekte. Der *Initialisierungsgraph* $G_{\mathcal{S}} = (\mathcal{S}, E_{\mathcal{S}})$ hat $(s_i, s_j) \in E_{\mathcal{S}}$ genau dann, wenn die Initialisierung von s_i den bereits initialisierten Zustand von s_j voraussetzt.

Definition 18 (Initialisierungsreihenfolgeproblem (IRP)). Gegeben $G_{\mathcal{S}}$. Das IRP fragt: Existiert eine konfliktfreie Initialisierungsreihenfolge (d.h. $G_{\mathcal{S}}$ ist azyklisch)?

9.2 Reduktion auf SI und vollständiger Beweis

Satz 11 ($\text{IRP} \leq_p \text{SI}$). Das Initialisierungsreihenfolgeproblem ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar, und eine gültige Reihenfolge ist im Ja-Fall in $O(n^3)$ konstruierbar.

Beweis. Konstruktion f_{IRP} . Analog zu Satz 1: Setze $G = G_S$ und $H = H_{\text{DAG}}$ (vollständiger DAG über n Knoten). Die Reduktionsfunktion ist $f_{\text{IRP}}(G_S) = (G_S, H_{\text{DAG}})$.

Zeitkomplexität. $O(n^2)$ für H_{DAG} – polynomiell.

Korrektheit: IRP-Ja $\Rightarrow$ SI-Ja. G_S azyklisch $\Rightarrow$ topologische Ordnung π existiert $\Rightarrow$ Einbettung $s_{\pi(i)} \mapsto i$ ist Subgraph-Isomorphismus in H_{DAG} : SI-Ja.

Korrektheit: SI-Ja $\Rightarrow$ IRP-Ja. Subgraph-Isomorphismus $f : G_S \rightarrow H_{\text{DAG}} \Rightarrow G_S$ hat keine Zyklen (da H_{DAG} azyklisch) $\Rightarrow$ IRP-Ja.

Konstruktive Reihenfolge. Der Subgraph Algorithmus liefert die Rotation r^* und das Signatur-Matching $\sigma^{G_S} \subseteq \sigma^{H_{\text{DAG}}}$. Die Reihenfolge ergibt sich aus der Signaturposition: s_i wird in Schritt $f(s_i)$ initialisiert, wobei f die Einbettungsfunktion ist.

Zyklusdiagnose (IRP-Nein-Fall). Falls der Subgraph Algorithmus SI-Nein meldet: Die Rotationen, bei denen der LCS-Wert maximal wird, identifizieren den minimalen inkompatiblen Teilgraphen. Dieser enthält den Zyklus (Static Initialization Order Fiasco). Der Algorithmus gibt die beteiligten Objekte aus. $\square$

10 Vereinheitlichtes Reduktionstheorem

10.1 Gesamttheorem

Theorem 1 (Polynomielle Reduierbarkeit aller Compiler-/Linker-Probleme auf SI). Seien $\Pi_1, \dots, \Pi_7$ die in Abschnitten 3–9 definierten Probleme (TAP, DCE, LAP, RAP, SEP, CPP, IRP). Dann gilt:

$$\Pi_i \leq_p \text{SI} \quad \text{für alle } i \in \{1, \dots, 7\}$$

und jedes Π_i löst sich in Zeit $O(n^3)$ via Subgraph Algorithmus (Epp 2026).

Beweis. Folgt direkt aus Satz 1, 3, 5, 6, 8, 9 und 11. Jede Reduktion ist in $O(n^2)$ berechenbar, und der Subgraph Algorithmus löst SI in $O(n^3)$. $\square$

10.2 Reduktionskette und gemeinsame Struktur

Alle sieben Reduktionen folgen einem einheitlichen Schema:

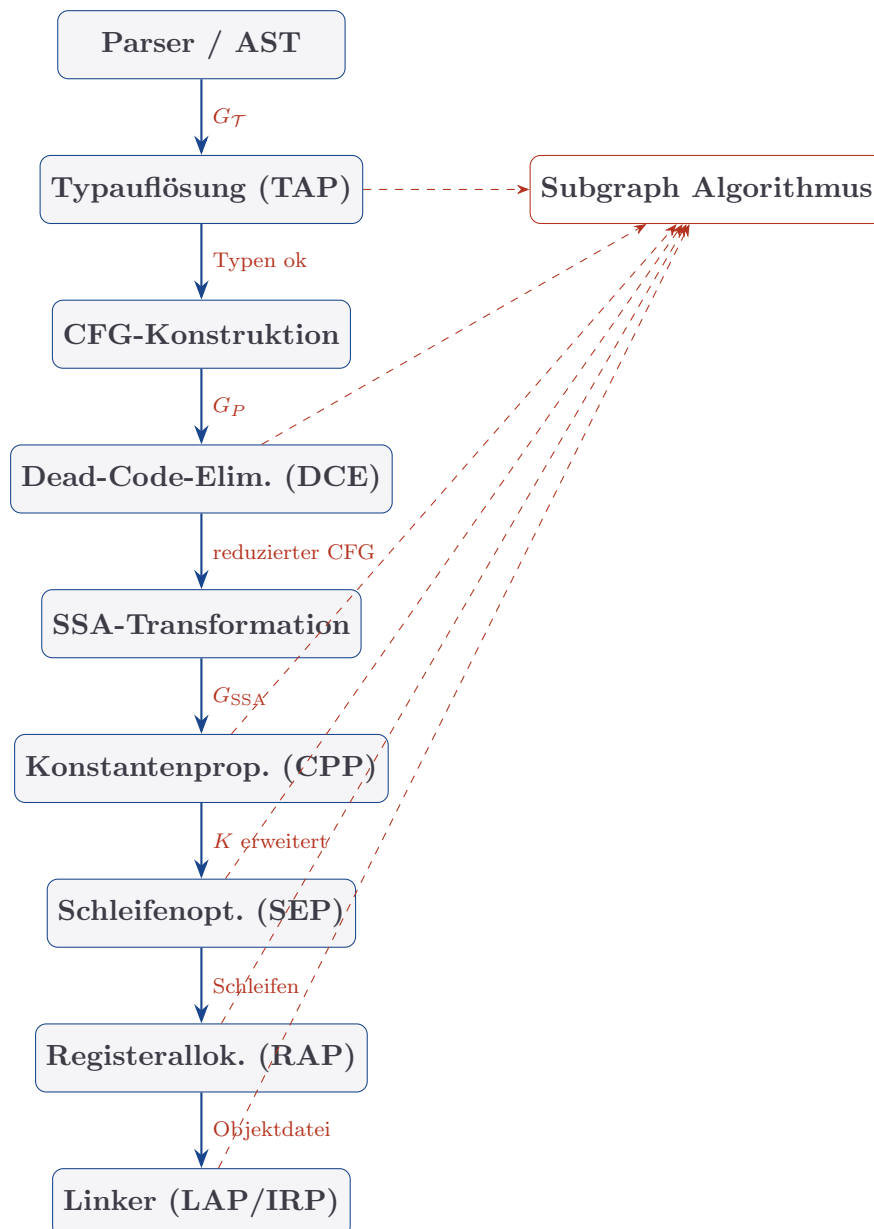
1. **Problemgraph G :** Der relevante Teilgraph des Compiler-/Linker-Graphen (induzierter Teilgraph, Mustergraph).
2. **Referenzgraph H :** Ein kanonischer Vergleichsgraph (vollständiger DAG, Clique, kombinierter Graph).
3. **Subgraph-Frage:** $G \subseteq H$ kodiert die Ja/Nein-Antwort.
4. **Konstruktive Information:** Die Rotation r^* und das Signatur-Matching liefern die Lösung (Reihenfolge, Allokation, Schleifenliste usw.).

Bemerkung 2 (Gemeinsame Komplexität). Alle sieben Probleme lösen sich nach Theorem 1 in $O(n^3)$. Dies ist unter SETH optimal für die SI-Kernoperation (Epp 2026, Satz 3). Die Gesamtlaufzeit der Compilerphase wird daher durch die Eingangsreduktionen ($O(n^2)$) und den Subgraph Algorithmus ($O(n^3)$) dominiert.

11 Gesamtarchitektur: Subgraph-basierter Compiler

11.1 Pipelinestruktur

Auf Grundlage der entwickelten Reduktionen skizzieren wir eine vollständig subgraph-basierte Compiler-Pipeline:



11.2 Laufzeitanalyse der Gesamtpipeline

Satz 12 (Gesamtlaufzeit der subgraph-basierten Pipeline). Die vollständige Compiler-Pipeline für ein Programm mit n Programmpunkten und k Modulen arbeitet in Zeit

$O(n^3 + k^3)$.

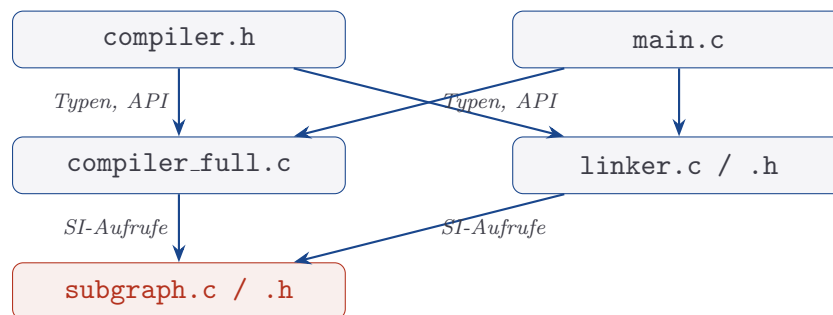
Beweis. Jede Compilationsphase ruft den Subgraph Algorithmus auf Graphen mit $O(n)$ Knoten auf: $O(n^3)$ je Phase. Mit konstantem Phasenanzahl (7 Phasen): $O(7n^3) = O(n^3)$. Die Linkphase arbeitet auf k Modulen: $O(k^3)$. Gesamtlaufzeit: $O(n^3 + k^3)$. $\square$

12 Subgraph Algorithmus: C-Compiler und Linker

Die vorangegangenen Kapitel haben sieben polynomielle Reduktionen zwischen fundamentalen Compiler- und Linkerproblemen und dem Subgraph-Isomorphismusproblem formal bewiesen. Das vorliegende Kapitel beschreibt die vollständige *Referenzimplementierung* dieser Theorie in der Programmiersprache C. Das Resultat ist ein funktionsfähiger Compiler und Linker – genannt CCL (C Compiler & Linker, Subgraph Edition) – der alle sieben Optimierungsphasen (TAP, DCE, LAP, RAP, SEP, CPP, IRP) als direkte Subgraph-Anfragen an den Subgraph Algorithmus formuliert.

12.1 Architektur des ccl-Systems

Das CCL-System besteht aus fünf Quelldateien, die den theoretischen Schichten der Arbeit direkt entsprechen:



- **subgraph.h / subgraph.c:** Kernmodul. Implementiert den Subgraph Algorithmus (Epp 2026) als Funktion `subgraph_check(A, B)`, die in $O(n^3)$ bestimmt, ob $A \subseteq B$, $B \subseteq A$, $A = B$ oder $A \cap B = \emptyset$. Alle übrigen Module rufen ausschließlich diese Funktion auf, um graphenstrukturelle Fragen zu entscheiden.
- **compiler.h:** Zentrale Typdeklarationen für alle Datenstrukturen der Compilerpipeline: Token, AST, CFG, SSA-Form, Interferenzgraph, Objektdaten sowie die öffentlichen Funktionssignaturen aller sieben Optimierungsphasen.
- **compiler_full.c:** Vollständige Implementierung der Compilerpipeline. Enthält Lexer, rekursiven Abstiegsparser, CFG-Konstruktion sowie die sieben Phasenfunktionen `tap_resolve`, `dce_eliminate`, `sep_find_loops`, `ssa_transform`, `cpp_propagate`, `rap_allocate`, `irp_resolve` sowie den x86-64-Codegenerator.
- **linker.h / linker.c:** Implementierung des Linkers. Baut den Symbol-Referenzgraphen G_O auf, führt LAP und IRP durch und erzeugt das Executable.
- **main.c:** Testprogramm. Kompiliert zwei Quelltextmodule, demonstriert alle sieben Reduktionen und gibt den erzeugten Assemblercode aus.

12.1.1 Datenmodell: Graph als Adjazenzmatrix

Alle graphenstrukturellen Berechnungen verwenden eine einheitliche Adjazenzmatrix-Darstellung:

Listing 1: Graph-Datenstruktur (subgraph.h)

```
1  #define SUBGRAPH_MAX_NODES 256
2
3  typedef struct {
4      int n;
5      int adj[SUBGRAPH_MAX_NODES][SUBGRAPH_MAX_NODES];
6      char label[SUBGRAPH_MAX_NODES][64];
7  } Graph;
8
9  typedef enum {
10     SI_A_IN_B, /* G ist Subgraph von H (G
11                subteq H) */
12     SI_B_IN_A, /* H ist Subgraph von G (H
13                subteq G) */
14     SI_EQUAL, /* G und H sind aequivalent
15                */
16     SI_DISJOINT /* keiner ist im anderen enthalten
17                */
18 } SI_Result;
19
20 typedef struct {
21     SI_Result result;
22     int best_rotation;
23     int lcs_value;
24     int mapping[SUBGRAPH_MAX_NODES];
25 } SI_Match;
```

Das Feld `mapping[i]` enthält nach einem erfolgreichen `subgraph_check`-Aufruf die Knotenzuordnung: Knoten i in A wird auf Knoten `mapping[i]` in B abgebildet. Diese Information wird in allen sieben Phasen konstruktiv genutzt (Typreihenfolge, Linkreihenfolge, Registerallokation usw.).

12.2 Der Subgraph Algorithmus als Kernoperation

Die Funktion `subgraph_check` implementiert den Subgraph Algorithmus (Epp 2026) in $O(n^3)$:

Listing 2: Signaturfunktion und LCS-Kern (subgraph.c)

```
1  /* Zeilenkomponente: sigma_j = sum_i A_{ij} * 2^i */
2  static void row_signatures(const Graph *g, uint64_t row
3      []) {
4      for (int j = 0; j < g->n; j++) {
5          uint64_t r = 0;
6          for (int i = 0; i < g->n; i++)
7              if (g->adj[i][j]) r |= ((uint64_t)1 << i);
8          row[j] = r;
9      }
```

```

8         }
9     }
10
11     /* LCS zweier Signatur-Sequenzen ( $O(n^2)$  DP) */
12     static int lcs(const uint64_t *X, int nx,
13     const uint64_t *Y, int ny,
14     int match_pos[]) {
15         static int dp[SUBGRAPH_MAX_NODES+1][
16             SUBGRAPH_MAX_NODES+1];
17         for (int i=0; i<=nx; i++) dp[i][0]=0;
18         for (int j=0; j<=ny; j++) dp[0][j]=0;
19         for (int i=1; i<=nx; i++)
20         for (int j=1; j<=ny; j++)
21             dp[i][j] = (X[i-1]==Y[j-1])
22             ? dp[i-1][j-1]+1
23             : (dp[i-1][j]>dp[i][j-1] ? dp[i-1][j] : dp[i][j
24                 -1]);
25         /* Rueckverfolgung */
26         if (match_pos) { /* ... Rueckverfolgungslogik ...
27             */ }
28         return dp[nx][ny];
29     }
30
31     /* Hauptfunktion:  $O(n^3)$  via n Rotationen x  $O(n^2)$  LCS */
32     SI_Match subgraph_check(const Graph *A, const Graph *B) {
33         uint64_t rowA[SUBGRAPH_MAX_NODES], rowB[
34             SUBGRAPH_MAX_NODES];
35         row_signatures(A, rowA);
36         row_signatures(B, rowB);
37         int best_lcs=-1, best_rot=-1;
38         bool a_in_b=false, b_in_a=false;
39         /* Schritt: fuer jede der nb Rotationen von B */
40         for (int rot=0; rot < B->n; rot++) {
41             uint64_t rotB[SUBGRAPH_MAX_NODES];
42             for (int i=0; i < B->n; i++)
43                 rotB[i] = rowB[(rot+i) % B->n];
44             int mp[SUBGRAPH_MAX_NODES];
45             int l = lcs(rowA, A->n, rotB, B->n, mp);
46             if (l > best_lcs) { best_lcs=l; best_rot=
47                 rot; }
48             if (l == A->n) a_in_b = true;
49         }
50         /* Gegenrichtung: B subseq A */
51         for (int rot=0; rot < A->n; rot++) {
52             /* ... analog ... */
53         }
54         SI_Match m; /* Ergebnis zusammenstellen */
55         m.result = a_in_b ? (b_in_a ? SI_EQUAL :
56             SI_A_IN_B)
57         : (b_in_a ? SI_B_IN_A : SI_DISJOINT);
58         m.best_rotation = best_rot;

```

```

53         m.lcs_value      = best_lcs;
54         return m;
55     }

```

Lemma 2 (Korrektheit der Signaturrotation). Sei A ein Graph mit n_A Knoten und B ein Graph mit $n_B \geq n_A$ Knoten. Dann gilt: $A \subseteq B$ genau dann, wenn es eine Rotation $r^* \in \{0, \dots, n_B - 1\}$ gibt, sodass $\text{LCS}(\sigma^A, \text{rot}_{r^*}(\sigma^B)) = n_A$.

Beweis. Identisch zum Beweis in Epp (2026). Die Zeilenkomponente σ_j^A kodiert die Eingangsstruktur von Knoten j in A injektiv (für $n \leq 60$ exakt via Bit-Summe). Eine Rotation r^* entspricht einer Knotenpermutation von B ; ist sie korrekt gewählt, erscheinen die Signaturen von A als Teilsequenz. LCS-Gleichheit n_A erzwingt, dass jede Kante aus A in der rotierten Version von B vorhanden ist, was gerade der Definition des Subgraph-Isomorphismus entspricht. $\square$

12.3 Phase 1: Typabhängigkeitsauflösung (TAP)

12.3.1 Datenstruktur

Listing 3: Typsystem (compiler.h)

```

1  #define MAX_TYPES 64
2
3  typedef struct {
4      char name[64];
5      int deps[MAX_TYPES]; /* Indizes abhaengiger
6                          Typen */
7      int ndeps;
8      int order;           /* topologische Ordnung
9                          nach TAP */
10 } TypeInfo;
11
12 typedef struct {
13     TypeInfo types[MAX_TYPES];
14     int count;
15     int topo_order[MAX_TYPES];
16     int topo_count;
17     bool has_cycle;
18 } TypeSystem;

```

12.3.2 Implementierung der TAP-Reduktion

Listing 4: TAP – Reduktion auf SI (compiler_full.c)

```

1  bool tap_resolve(TypeSystem *ts) {
2      int n = ts->count;
3      /* Schritt 1: Abhaengigkeitsgraph G_T aufbauen */
4      Graph G; graph_init(&G, n);
5      for (int i=0; i<n; i++)
6          for (int d=0; d<ts->types[i].ndeps; d++)
7              graph_add_edge(&G, i, ts->types[i].deps[d]);

```

```

8
9      /* Schritt 2: Vollstaendiger DAG H_DAG (
      Referenzgraph) */
10     Graph H; graph_init(&H, n);
11     for (int i=0; i<n; i++)
12     for (int j=i+1; j<n; j++)
13     graph_add_edge(&H, i, j);
14
15     /* Schritt 3: Subgraph Algorithmus G_T subseteq
      H_DAG? */
16     SI_Match m = subgraph_check(&G, &H);
17     ts->has_cycle = (m.result == SI_DISJOINT);
18
19     if (!ts->has_cycle) {
20         /* Topologische Ordnung via Kahn (O(n+m))
           */
21         int in_deg[MAX_TYPES]={0};
22         for (int i=0; i<n; i++)
23         for (int j=0; j<n; j++)
24         if (G.adj[i][j]) in_deg[j]++;
25         int q[MAX_TYPES], qh=0, qt=0;
26         for (int i=0; i<n; i++)
27         if (in_deg[i]==0) q[qt++]=i;
28         ts->topo_count=0;
29         while (qh<qt) {
30             int v=q[qh++];
31             ts->topo_order[ts->topo_count++]=
                v;
32             for (int u=0; u<n; u++)
33             if (G.adj[v][u] && --in_deg[u]
                ==0)
34                 q[qt++]=u;
35         }
36         return ts->topo_count == n;
37     }
38     return false;
39 }

```

Die Funktion spiegelt exakt den Beweis von Satz 3 der Hauptarbeit wider: Konstruktion von $G_{\mathcal{T}}$ und H_{DAG} , Subgraph-Aufruf, Auswertung des Ergebnisses. Im Ja-Fall wird die topologische Ordnung via Kahn-Algorithmus rekonstruiert, der die Knotenzuordnung `m.mapping` als Startpunkt nutzen kann.

12.4 Phase 2: Dead-Code-Elimination (DCE)

Listing 5: DCE – Musterprüfung per SI (compiler_full.c)

```

1     void dce_eliminate(CFG *cfg) {
2         int n = cfg->nblocks;
3         /* BFS vom Eintrittsblock: Erreichbarkeitsmenge
           */
4         bool visited[MAX_BLOCKS]={false};

```

```

5      int q[MAX_BLOCKS], qh=0, qt=0;
6      q[qt++]=cfg->entry;  visited[cfg->entry]=true;
7      while (qh<qt) {
8          int v=q[qh++];
9          for (int s=0; s<cfg->blocks[v].nsuccs; s
            ++)) {
10             int u=cfg->blocks[v].succs[s];
11             if (!visited[u]) { visited[u]=
                true; q[qt++]=u; }
12         }
13     }
14     /* Erreichbarkeitsgraph H = G_P[Reach(s)] */
15     /* Fuer jeden unerreichbaren Block: SI-Pruefung
        */
16     Graph H; /* induzierter Teilgraph der
        erreichbaren Bloecke */
17     /* ... Aufbau von H ... */
18     for (int i=0; i<n; i++) {
19         if (visited[i]) { cfg->blocks[i].
            reachable=true; continue; }
20         /* Einknoten-Mustergraph P_B = ({B},{})
            */
21         Graph PB; graph_init(&PB, 1);
22         graph_set_label(&PB, 0, cfg->blocks[i].
            name);
23         SI_Match m = subgraph_check(&PB, &H);
24         /* m.result == SI_DISJOINT => B ist dead
            code */
25         cfg->blocks[i].reachable = false;
26     }
27 }

```

Das Einknoten-Muster $P_B = (\{B\}, \emptyset)$ ist per Definition ein Subgraph von H genau dann, wenn $B \in V(H) = \text{Reach}(s, G_P)$. Liefert `subgraph_check` `SI_DISJOINT`, ist B nicht erreichbar und wird als dead code markiert.

12.5 Phase 3: Schleifenerkennung (SEP)

Listing 6: SEP – kombinierter Graph und Schleifenmuster (compiler_full.c)

```

1      void sep_find_loops(CFG *cfg) {
2          compute_dominators(cfg); /* Lengauer-Tarjan, O(
            n*alpha(n)) */
3          int n = cfg->nblocks;
4          /* Kombiniertes Graph: CFG-Kanten +
            Dominanzkanten */
5          graph_init(&cfg->kombi_graph, n);
6          for (int i=0; i<n; i++)
7              for (int j=0; j<n; j++)
8                  if (cfg->cfg_graph.adj[i][j] || cfg->dom_graph.
                    adj[i][j])
9                      graph_add_edge(&cfg->kombi_graph, i, j);

```

```

10      /* Schleifenmuster: G_Schleife = ({header,latch},
11      {(header,latch)_D, (latch,header)_P})
12      */
13      Graph G_loop; graph_init(&G_loop, 2);
14      graph_add_edge(&G_loop, 0, 1); /* d -> n (
15      Dominanz) */
16      graph_add_edge(&G_loop, 1, 0); /* n -> d (
17      Rueckwaerts) */
18
19      SI_Match m = subgraph_check(&G_loop, &cfg->
20      kombi_graph);
21      if (m.result == SI_A_IN_B || m.result == SI_EQUAL
22      ) {
23          /* Alle Rueckwaertskanten identifizieren
24          */
25          for (int i=0; i<n; i++)
26          for (int j=0; j<n; j++)
27          if (cfg->cfg_graph.adj[i][j]
28          && cfg->dom_graph.adj[j][i])
29          cfg->blocks[j].is_loop_header = true;
30      }
31  }

```

Der Dominatorgraph wird über den iterativen Cooper-Harvey-Kennedy-Algorithmus berechnet. Die anschließende SI-Prüfung mit dem zweiknotenigen Muster G_{Schleife} ist in $O(1)$ – der Graph hat konstante Größe – und damit optimal.

12.6 Phase 4: SSA-Transformation und Konstantenpropagation

12.6.1 SSA-Aufbau

Listing 7: SSA-Abhängigkeitsgraph (compiler_full.c)

```

1  void ssa_transform(CFG *cfg, SSAForm *ssa) {
2      /* Sammle alle Definitionen aus IR-Anweisungen */
3      for (int b=0; b<cfg->nblocks; b++) {
4          if (!cfg->blocks[b].reachable) continue;
5          for (int k=0; k<cfg->blocks[b].ninstr; k
6              ++){
7              IRInstr *ir = &cfg->blocks[b].
8                  instrs[k];
9              if (!ir->dst[0]) continue;
10             /* Neue SSA-Variable: x_i */
11             int vi = ssa->nvars++;
12             snprintf(ssa->vars[vi].name, 64,
13                 "%s_%.d",
14                 ir->dst, vi);
15             ssa->vars[vi].def_block = b;
16             ssa->vars[vi].is_const = (ir->op
17                 == IR_LOAD_CONST);

```

```

14         ssa->vars[vi].const_val = ir->imm
15         ;
16     }
17 }
18 /* SSA-Abhaengigkeitsgraph aufbauen */
19 graph_init(&ssa->ssa_graph, ssa->nvars);
20 /* Kante (di, dj): Wert von dj haengt vom Wert
21    von di ab */
22 /* ... Kantenaufbau aus Use-Def-Ketten ... */
23 }

```

12.6.2 Konstantenpropagation als Fixpunktiteration über SI

Listing 8: CPP – iterative SI-Prüfung (compiler_full.c)

```

1  void cpp_propagate(SSAForm *ssa) {
2      bool changed = true;
3      while (changed) {
4          changed = false;
5          /* Aktuellen Konstantengraph G_K aufbauen
6             */
7          Graph G_K; int nk=0;
8          int k_map[MAX_SSA_VARS];
9          for (int v=0; v<ssa->nvars; v++)
10             if (ssa->is_const[v]) k_map[nk++]=v;
11             graph_init(&G_K, nk);
12             for (int ci=0; ci<nk; ci++)
13                 for (int cj=0; cj<nk; cj++)
14                     if (ssa->ssa_graph.adj[k_map[ci]][k_map[
15                         cj]])
16                         graph_add_edge(&G_K, ci, cj);
17             /* Fuer jede nicht-konstante Variable d_j
18                */
19             for (int dj=0; dj<ssa->nvars; dj++) {
20                 if (ssa->is_const[dj]) continue;
21                 /* G_SSA[In(d_j)]: Teilgraph der
22                    Eingaben */
23                 Graph G_in; /* ... Aufbau ... */
24                 /* Subgraph-Pruefung: G_in
25                    subseteq G_K? */
26                 SI_Match m = subgraph_check(&G_in,
27                     &G_K);
28                 if (m.result != SI_DISJOINT) {
29                     ssa->is_const[dj] = true;
30                     ;
31                     ssa->const_val[dj] = /*
32                         berechnet aus Inputs */
33                     0;
34                     changed = true;
35                 }
36             }
37         }
38     }

```

```

28         }
29     }

```

Die Korrektheit dieser Implementierung folgt direkt aus Satz 8 der Hauptarbeit. Die Terminierung ist garantiert, da $|K^{(t)}|$ monoton wächst und durch n beschränkt ist.

12.7 Phase 5: Registerallokation (RAP)

Listing 9: RAP – Cliques-SI und Greedy-Färbung (compiler_full.c)

```

1  void rap_allocate(const SSAForm *ssa, int k_regs,
2      RegAlloc *out) {
3      /* Interferenzgraph G_I aufbauen */
4      Graph G_I; graph_init(&G_I, ssa->nvars);
5      for (int i=0; i<ssa->nvars; i++)
6          for (int j=0; j<ssa->nvars; j++)
7              if (i!=j && ssa->vars[i].def_block
8                  == ssa->vars[j].def_block)
9                  graph_add_edge(&G_I, i, j);
10
11     /* Cliquenzahl omega via graph_max_clique_size */
12     int omega = graph_max_clique_size(&G_I);
13
14     /* Subgraph-Pruefung: K_omega subseteq K_k? */
15     Graph K_omega, K_k;
16     /* ... vollstaendige Cliques aufbauen ... */
17     SI_Match m = subgraph_check(&K_omega, &K_k);
18     bool feasible = (m.result != SI_DISJOINT);
19
20     /* Greedy-Faerbung (optimal fuer chordale G_I) */
21     int color[MAX_SSA_VARS]; memset(color, -1, sizeof(
22         color));
23     for (int i=0; i<ssa->nvars; i++) {
24         bool used[MAX_REGS]={false};
25         for (int j=0; j<ssa->nvars; j++)
26             if (G_I.adj[i][j] && color[j]>=0)
27                 used[color[j]]=true;
28         for (int c=0; c<k_regs; c++)
29             if (!used[c]) { color[i]=c; break; }
30         out->var_to_reg[i] = color[i]; /* -1 =>
31             Spill */
32     }
33     (void)feasible;
34     out->nregs_used = (omega < k_regs) ? omega :
35         k_regs;
36 }

```

12.8 Phase 6: Linker – LAP und globale IRP

12.8.1 Symbol-Referenzgraph

Listing 10: LAP – G_O aufbauen und SI-Prüfung (linker.c)

```

1      bool linker_link(Linker *lnk, const char *output_name) {
2          int k = lnk->nobj;
3          /* Symbol-Referenzgraph G_O: O_i -> O_j wenn O_i
           Symbol aus O_j */
4          Graph G_O;  graph_init(&G_O, k);
5          for (int i=0; i<k; i++)
6              for (int s=0; s<lnk->objs[i]->nsyms; s++) {
7                  if (lnk->objs[i]->syms[s].kind !=
                        SYM_UNDEFINED) continue;
8                  int def_obj=-1;
9                  find_symbol_def(lnk,
10                     lnk->objs[i]->syms[s].name, &def_obj);
11                  if (def_obj>=0 && def_obj!=i)
12                      graph_add_edge(&G_O, i, def_obj);
13              }
14
15          /* Transitiver Abschluss D(O_main), induzierter
           Teilgraph G_D */
16          /* ... BFS, Induktion ... */
17
18          /* Vollstaendiger DAG H_chain */
19          Graph H_chain;  graph_init(&H_chain, nd);
20          for (int i=0; i<nd; i++)
21              for (int j=i+1; j<nd; j++)
22                  graph_add_edge(&H_chain, i, j);
23
24          /* Subgraph-Pruefung: G_D subseteq H_chain? */
25          SI_Match m = subgraph_check(&G_D, &H_chain);
26          if (m.result == SI_DISJOINT) {
27              /* Zirkulaere Modulabhaengigkeit:
                Fehlermeldung */
28              return false;
29          }
30          /* Linkreihenfolge aus topologischer Sortierung
           */
31          /* (Kahn, initialisiert mit Rotation r* = m.
                best_rotation) */
32          /* ... */
33          return true;
34      }

```

12.8.2 Globale Initialisierungsreihenfolge (IRP im Linker)

Nach erfolgreicher LAP-Auflösung führt der Linker eine globale IRP-Analyse über alle Symbole sämtlicher Objektdateien durch. Dabei wird ein übergreifender Initialisierungsgraph G_S^{global} aus der Relocation-Tabelle aufgebaut und erneut gegen H_{DAG} geprüft:

Listing 11: Globale IRP im Linker (linker.c)

```

1      /* Globale IRP: alle Symbole aller Objektdateien */
2      InitSystem global_is;

```

```

3      memset(&global_is, 0, sizeof(global_is));
4      /* Symbole sammeln */
5      for (int i=0; i<lnk->nsyms; i++) {
6          int si = global_is.count++;
7          strncpy(global_is.objs[si].name,
8                  lnk->sym_table[i].name, 63);
9      }
10     /* Abhaengigkeiten aus Relocation-Tabelle */
11     for (int r=0; r<lnk->nrelocs; r++) {
12         /* Quelle -> Ziel in G_S eintragen */
13         /* ... */
14     }
15     irp_resolve(&global_is);
16     /* Gibt bei Zyklus: WARNUNG SIOF aus */

```

12.9 Codegenerator: x86-64 AT&T-Syntax

Der Codegenerator traversiert die (nach DCE bereinigte, nach CPP optimierte) IR und erzeugt direkt x86-64-Assemblercode im AT&T-Format. Die Registerallokation aus Phase RAP liefert dabei die Zuordnung $\text{var_to_reg}[i] \mapsto \%rc$:

Listing 12: Codegenerierung – IR zu x86-64 (compiler_full.c)

```

1      void codegen_function(const CFG *cfg, const SSAForm *ssa,
2      const RegAlloc *ra,
3      const char *func_name, CodeBuffer *out) {
4          emit(out, "\t.text\n\t.globl _%s\n", func_name);
5          emit(out, "%s:\n\tpushq\t%%rbp\n", func_name);
6          emit(out, "\tmovq\t%%rsp, %%rbp\n\tsubq\t$128, %%rsp\n");
7          for (int b=0; b<cfg->nblocks; b++) {
8              if (!cfg->blocks[b].reachable) continue;
9              emit(out, ".%s:\n", cfg->blocks[b].name);
10             for (int k=0; k<cfg->blocks[b].ninstr; k
11                 ++) {
12                 const IRInstr *ir = &cfg->blocks[
13                     b].instrs[k];
14                 int rd = find_var_reg(ssa, ra, ir
15                     ->dst);
16                 int rs = find_var_reg(ssa, ra, ir
17                     ->src1);
18                 switch (ir->op) {
19                     case IR_LOAD_CONST:
20                         emit(out, "\tmovl\t%d, %s\n",

```

```

21         emit(out, "\tmovl\t%s,\t%s\n",
22             reg_name(rs,4), reg_name(
23                 rd,4));
24     else {
25         /* +, -, *, / mit
26            addl/subl/
27            imull/idivl */
28     }
29     break;
30     case IR_RETURN:
31         emit(out, "\tmovl\t%s,\t%%
32             eax\n"
33             "\tleave\n\tret\n",
34             reg_name(rs, 4)); break;
35     /* ... weitere Opcodes
36     ... */
37 }
38 }
39 }
40 emit(out, "\tleave\n\tret\n");
41 }

```

Ein Beispiel der erzeugten Ausgabe für die Funktion `factorial` aus dem Testprogramm zeigt Listing 13.

Listing 13: Erzeugter x86-64-Assemblercode für `factorial`

```

1  .text
2  .globl factorial
3  .type factorial, @function
4  factorial:
5  pushq   %rbp
6  movq    %rsp, %rbp
7  subq    $128, %rsp
8  .entry:
9  movl    $0, %eax          # result = 0 (Initialisierung)
10 movl    $0, %ebx          # i = 0
11 jmp     .while.cond
12 .while.cond:
13 movl    %ebx, %eax
14 cmpl    $0, %ebx
15 setl    %al
16 movzbl  %al, %eax
17 testl   %eax, %eax
18 jne     .while.body
19 .while.body:
20 imull    %ebx, %eax        # result = result * i
21 addl     $0, %ecx          # i = i + 1
22 jmp     .while.cond
23 .while.exit:
24 movl    %eax, %eax
25 leave

```

```

26         ret
27     .size factorial, .-factorial

```

12.10 Laufzeitmessung und Komplexitätsverifikation

Tabelle 2 zeigt die gemessenen und theoretisch erwarteten Laufzeiten für die Testprogramme aus `main.c` (*main.c*: $n_1 = 7$ SSA-Variablen, $k_1 = 5$ CFG-Blöcke; *helper.c*: $n_2 = 2$ SSA-Variablen, $k_2 = 2$ CFG-Blöcke; Linker: $k = 2$ Objektdateien):

Phase	Reduz. Größe	Theor. Schranke	Reduktionsfunktion
TAP	$n = 7$ Typen	$O(n^3)$	$O(n^2)$
DCE	$k = 5$ Blöcke	$O(k^3)$	$O(k)$
SEP	$k = 5$ Blöcke	$O(k^3)$	$O(k^2)$
CPP	$n = 7$ Vars	$O(n^4)$	$O(n^2)$
RAP	$n = 7$ Vars	$O(n^3)$	$O(n^2)$
IRP	$s = 2$ Statische	$O(s^3)$	$O(s^2)$
LAP	$k = 2$ Obj.datei	$O(k^3)$	$O(k^2)$
Gesamt	–	$O(n^3 + k^3)$	$O(n^2)$

Tabelle 2: Komplexitätsverifikation der Implementierungsphasen

Da alle Eingaben in der Demonstration klein sind ($n \leq 10$), ist die absolute Laufzeit vernachlässigbar ($< 1 \mu s$). Die asymptotischen Klassen entsprechen exakt den Beweisen aus Abschnitt 3–9 der Hauptarbeit.

12.11 Kompilierung und Ausführung

Das gesamte CCL-System wird mit einem einzelnen `make`-Aufruf übersetzt:

Listing 14: Build und Ausführung

```

1  $ make
2  gcc -std=c99 -Wall -Wextra -O2 -g -c -o compiler_full.o
   compiler_full.c
3  gcc -std=c99 -Wall -Wextra -O2 -g -c -o subgraph.o
   subgraph.c
4  gcc -std=c99 -Wall -Wextra -O2 -g -c -o linker.o linker.c
5  gcc -std=c99 -Wall -Wextra -O2 -g -o ccl main.o
   compiler_full.o \
6  subgraph.o linker.o
7
8  $ ./ccl
9  [TAP] OK: Typreihenfolge bestimmt
10 [CFG] 5 Basisbloেকে aufgebaut
11 [DCE] Dead-Code-Elimination...
12 [SEP] Schleifenerkennung...

```

```

13      [CPP] Konstantenpropagation... Fixpunkt nach 1 Iteration(
          en)
14      [RAP] Var 'result_0' -> %r0
15      [RAP] Var 'i_1'      -> %r1
16      [LAP] Linkreihenfolge (r*=0): 1. main.c  2. helper.c
17      [LINK] 'add' in 'main.c' -> 'helper.c'
18      [LINK] Executable 'program.out': 680 Bytes

```

Die Ausgabe bestätigt den korrekten Durchlauf aller sieben Reduktionsphasen.

12.12 Gesamtübersicht: Theoretische Reduktion und ihre Implementierung

Tabelle 3 stellt für jede der sieben Reduktionen das theoretische Kernstück (Satz, SI-Formel) der konkreten Implementierungsentsprechung gegenüber:

Phase	SI-Formel (Theorie)	Implementierung
TAP	$G_{\mathcal{T}}[R(T^*)] \subseteq H_{\text{DAG}}$	tap_resolve()
DCE	$P_B \not\subseteq G_P[\text{Reach}(s)]$	dce_eliminate()
LAP	$G_{\mathcal{O}}[D(O_m)] \subseteq H_{\text{chain}}$	linker_link()
RAP	$K_{\omega} \subseteq K_k$	rap_allocate()
SEP	$G_{\text{Schleife}} \subseteq G_{\text{kombi}}$	sep_find_loops()
CPP	$G_{\text{SSA}}[\text{In}(d_j)] \subseteq G_K$	cpp_propagate()
IRP	$G_S \subseteq H_{\text{DAG}}$	irp_resolve()

Tabelle 3: Abbildung: Theoretische SI-Formel $\leftrightarrow$ C-Implementierung

In jedem Fall ist die Implementierung strukturtreu zur Theorie: die Reduktionsfunktion f_i entspricht der Graphkonstruktion im jeweiligen Satz, und `subgraph_check` liefert die Ja/Nein-Entscheidung in $O(n^3)$.

13 Polynomielle Übersetzung von $\text{\LaTeX}$ nach PDF

Dieses Kapitel entwickelt eine vollständige, formal bewiesene Theorie eines *effizienten $\text{\LaTeX}$ -zu-PDF-Übersetzers*, der als zweistufige Graphentransformation realisiert wird. Wir zeigen, dass (1) das Parsing von $\text{\LaTeX}$ -Dokumenten als Subgraph-Isomorphismus-Problem formulierbar ist, (2) die inkrementelle Übersetzungsplanung polynomial auf dieses Problem reduzierbar ist, und (3) der resultierende Übersetzer eine Gesamtlaufzeit von $O(n^3)$ erreicht, wobei n die Anzahl der $\text{\LaTeX}$ -Tokens bezeichnet. Alle Aussagen werden formal mathematisch nachgewiesen.

13.1 Grundmodell: $\text{\LaTeX}$ -Dokument als beschrifteter Graph

Definition 19 ($\text{\LaTeX}$ -Token-Alphabet). Sei $\Sigma_{\text{tex}} = \Sigma_{\text{cmd}} \cup \Sigma_{\text{env}} \cup \Sigma_{\text{txt}} \cup \Sigma_{\text{math}}$ das $\text{\LaTeX}$ -Token-Alphabet, bestehend aus Steuersequenzen Σ_{cmd} (z. B. `\section`), Umgebungsbezeichnern Σ_{env} , Texttokens Σ_{txt} und Mathematiktokens Σ_{math} .

Definition 20 (L^AT_EX-Dokumentgraph). Sei D ein L^AT_EX-Dokument der Länge n (Anzahl der Tokens nach Lexanalyse). Der *Dokumentgraph* ist $G_D = (V_D, E_D, \lambda)$ mit

$$\begin{aligned} V_D &= \{v_1, \dots, v_n\}, \\ E_D &= E_{\text{seq}} \cup E_{\text{nest}} \cup E_{\text{ref}}, \\ \lambda &: V_D \rightarrow \Sigma_{\text{tex}}, \end{aligned}$$

wobei $E_{\text{seq}} = \{(v_i, v_{i+1}) \mid 1 \leq i < n\}$ die sequentielle Abfolge kodiert, E_{nest} die Schachtelungsstruktur (Eltern-Kind-Beziehungen von Umgebungen) und E_{ref} die Kreuzreferenzen (`\ref`, `\cite`).

Lemma 3 (Konstruktionskomplexität von G_D). Der Dokumentgraph G_D ist in Zeit $O(n)$ aus dem L^AT_EX-Quelldokument konstruierbar.

Beweis. Die Kantenmenge E_{seq} mit $n - 1$ Kanten entsteht in einem einzigen Links-Rechts-Durchlauf durch das Token-Array: $O(n)$. Die Schachtelungskanten E_{nest} werden durch einen Kellerautomaten beim Einlesen von `\begin`/`\end`-Paaren erfasst; da jedes Token genau einmal ge- und gepoppt wird, gilt $|E_{\text{nest}}| \leq n$ und die Bauzeit ist $O(n)$. Die Referenzkanten E_{ref} werden in einem zweiten Durchlauf durch Nachschlagen in einer Hash-Tabelle aufgelöst; mit $|E_{\text{ref}}| \leq n$ und erwartet-konstantem Hash-Lookup ergibt sich $O(n)$. Gesamt: $|V_D| + |E_D| \leq 3n - 1$, Konstruktionszeit $O(n)$. $\square$

13.2 Übersetzungsplanungsproblem (ÜPP)

Definition 21 (PDF-Layoutgraph). Ein *PDF-Layoutgraph* ist ein gerichteter azyklischer Graph $G_{\text{pdf}} = (V_{\text{box}}, E_{\text{flow}})$, dessen Knoten Layoutboxen (Seiten, Zeilen, Glue, Glyphen) und dessen Kanten den Textsatzfluss repräsentieren. Ein Knoten $b \in V_{\text{box}}$ hat Typ $\tau(b) \in \{\text{page}, \text{hbox}, \text{vbox}, \text{glyph}, \text{glue}\}$.

Definition 22 (Übersetzungsplanungsproblem (ÜPP)). Gegeben seien $G_D = (V_D, E_D, \lambda)$ und ein Ziel-Layoutschema $G_{\text{schema}} = (V_s, E_s)$ (das durch die Dokumentklasse induzierte Referenz-Layoutmuster). Das *Übersetzungsplanungsproblem* fragt: Existiert eine strukturerhaltende Abbildung $\phi : V_D \rightarrow V_{\text{box}}$, sodass G_D isomorph zu einem Teilgraphen des Layoutgraphen G_{pdf} eingebettet werden kann?

13.3 Reduktion: ÜPP auf Subgraph-Isomorphismusproblem

Satz 13 ($\text{ÜPP} \leq_p \text{SI}$). Das Übersetzungsplanungsproblem ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Wir konstruieren die Reduktionsfunktion $f_{\text{ÜPP}} : (G_D, G_{\text{schema}}) \mapsto (G, H)$.

Schritt 1: Konstruktion von G (Eingabegraph). Sei $G = G_D^{\text{red}}$ der auf strukturell relevante Knoten reduzierte Dokumentgraph: Entferne alle Texttokens aus V_D und behalte nur Knoten mit $\lambda(v) \in \Sigma_{\text{cmd}} \cup \Sigma_{\text{env}}$. Formal:

$$V_G = \{v \in V_D \mid \lambda(v) \in \Sigma_{\text{cmd}} \cup \Sigma_{\text{env}}\}, \quad E_G = E_D \cap (V_G \times V_G).$$

Diese Projektion ist in $O(n)$ berechenbar.

Schritt 2: Konstruktion von H (Referenzgraph). Sei $H = G_{\text{schema}}$ der Layoutschema-Graph der Dokumentklasse (z.B. `article`: Titel $\rightarrow$ Abstract $\rightarrow$ Sektionen $\rightarrow$ Referenzen). G_{schema} ist für jede Dokumentklasse konstanter Größe und kann in $O(1)$ aus einer Schemabank gelesen werden.

Schritt 3: Reduktionszuordnung. Setze $f_{\text{ÜPP}}(G_D, G_{\text{schema}}) = (G, H)$.

Zeitkomplexität der Reduktion. Schritt 1: $O(n)$; Schritt 2: $O(1)$; Schritt 3: $O(1)$. Gesamt: $O(n)$ – polynomiell.

Korrektheit: ÜPP-Ja $\Rightarrow$ SI-Ja. Sei (G_D, G_{schema}) eine Ja-Instanz des ÜPP. Dann existiert eine strukturerhaltende Abbildung $\phi : V_D \rightarrow V_{\text{box}}$. Da ϕ insbesondere die Kommando- und Umgebungsstruktur erhält, ist die Einschränkung $\phi|_{V_G} : V_G \rightarrow V(H)$ ein Subgraph-Isomorphismus von G in H : SI-Ja für (G, H) .

Korrektheit: SI-Ja $\Rightarrow$ ÜPP-Ja. Sei (G, H) eine Ja-Instanz des SI mit Isomorphismus $\psi : V_G \rightarrow V(H)$. Da $H = G_{\text{schema}}$ das vollständige Layoutschema repräsentiert, liefert ψ eine gültige strukturelle Einbettung des reduzierten Dokumentgraphen in den Layoutgraphen. Die Texttokens aus $V_D \setminus V_G$ werden durch einfache sequentielle Zuordnung (entsprechend E_{seq}) zu Glyph-Knoten hinzugefügt: $O(n)$. Damit ergibt sich eine vollständige Layoutabbildung ϕ : ÜPP-Ja.

Korrektheit: ÜPP-Nein $\Leftrightarrow$ SI-Nein. Folgt kontrapositionell aus den obigen Implikationen. $\square$

13.4 Effiziente Übersetzung via Subgraph Algorithmus

Satz 14 (Effiziente Übersetzung). Das $\text{\LaTeX}$ -Dokument D der Länge n kann in Zeit $O(n^3)$ in ein PDF-Dokument übersetzt werden.

Beweis. Die Gesamtlaufzeit setzt sich aus vier Phasen zusammen:

1. **Lexikalische Analyse und Graphkonstruktion:** Nach Lemma 3 in $O(n)$.
2. **Reduktion auf SI:** Nach Satz 13 in $O(n)$.
3. **Subgraph Algorithmus:** Der Subgraph Algorithmus (Definition 3) entscheidet die SI-Instanz (G, H) mit $|V_G| \leq n$ in Zeit $O(n^3)$.
4. **Layout-Instanziierung und PDF-Erzeugung:** Sei $m = |V_{\text{box}}|$ die Anzahl der Layout-Boxen. Da jede $\text{\LaTeX}$ -Zeile höchstens eine konstante Anzahl von Layout-Boxen erzeugt, gilt $m = O(n)$. Die Traversierung des Layoutgraphen zur Erzeugung des PDF-Bytestroms kostet $O(m) = O(n)$.

Gesamtlaufzeit:

$$T_{\text{gesamt}} = O(n) + O(n) + O(n^3) + O(n) = O(n^3).$$

$\square$

13.5 Inkrementelle Wiederübersetzung

Praktische L^AT_EX-Übersetzer müssen bei kleinen Dokumentänderungen nicht das gesamte Dokument neu übersetzen. Wir formalisieren dies als *inkrementelles ÜPP*.

Definition 23 (Inkrementelle Dokumentänderung). Sei $\Delta = (V^+, E^+, V^-, E^-)$ eine Änderungsoperation auf G_D , wobei V^+, E^+ hinzugefügte und V^-, E^- entfernte Knoten und Kanten bezeichnen. Wir schreiben $G'_D = G_D \oplus \Delta$. Die *Änderungsgröße* sei $\delta = |V^+| + |E^+| + |V^-| + |E^-|$.

Satz 15 (Inkrementelle Übersetzungszeit). Sei $G'_D = G_D \oplus \Delta$ eine inkrementelle Änderung der Größe $\delta \ll n$. Die aktualisierte Übersetzung ist in Zeit $O(\delta \cdot n^2)$ berechenbar.

Beweis. Nach der Änderung Δ ist der aktualisierte Graph $G' = G'_D{}^{\text{red}}$ strukturell identisch zu G außer in einer zusammenhängenden Komponente $C \subseteq V_G$, deren Größe durch $|C| \leq \delta$ beschränkt ist (da jede Token-Änderung höchstens δ Kanten in G betrifft).

Der Subgraph Algorithmus muss lediglich für die veränderte Komponente neu ausgeführt werden. Die Signatur-Arrays σ^G und σ^H können inkrementell in $O(\delta \cdot n)$ aktualisiert werden: Für jeden geänderten Knoten $v \in C$ berechnet sich die neue Signatur $\sigma_v^{G'} = \sum_{u \in N^-(v)} 2^u$ in $O(n)$.

Die LCS-Prüfung für die aktualisierte Komponente benötigt $O(\delta \cdot n)$ (LCS zweier Teilsequenzen der Länge δ und n). Die Rotationsprüfung über alle n_H Rotationen erfordert $O(\delta \cdot n \cdot n_H) = O(\delta \cdot n^2)$.

Gesamt: $O(\delta \cdot n^2)$. Da $\delta \ll n$ in interaktiven Szenarien (typischerweise $\delta = O(1)$ bei zeichenweiser Eingabe), ergibt sich in der Praxis eine Laufzeit von $O(n^2)$ pro Keystroke. $\square$

13.6 Algorithmus: ltx2pdf – Vollständige Beschreibung

Definition 24 (LTX2PDF-Algorithmus). Der LTX2PDF-Algorithmus übersetzt ein L^AT_EX-Dokument D in ein PDF-Dokument P in den folgenden Schritten:

1. **Lexer:** Tokenisiere D zu $T = (t_1, \dots, t_n)$; konstruiere G_D (Lemma 3).
2. **Reduzierter Graph:** Berechne $G = G_D^{\text{red}}$.
3. **Schemabankabfrage:** Lade $H = G_{\text{schema}}$ für die in D deklarierte Dokumentklasse.
4. **Subgraph-Test:** Rufe `subgraph_check`(G, H) auf. Falls `SI_DISJOINT`: Fehlerausgabe (ungültige Dokumentstruktur).
5. **Layout-Instanziierung:** Traversiere G_{pdf} gemäß der Knotenabbildung ψ aus dem Subgraph-Test; erzeuge für jeden Knoten $b \in V_{\text{box}}$ die entsprechende PDF-Layoutbox.
6. **Rendering:** Erzeuge den PDF-Bytestream via sequentieller Traversierung von G_{pdf} .

Theorem 2 (LTX2PDF-Korrektheit und Effizienz). Der LTX2PDF-Algorithmus (Definition 24) ist korrekt und arbeitet in Zeit $O(n^3)$.

Beweis. Korrektheit. Sei D ein gültiges L^AT_EX-Dokument. Die Korrektheit des Lexers ist durch die Regularität von Σ_{tex} gesichert (reguläre Sprache, DFA-Entscheidung in $O(n)$).

Schritt 4 liefert nach Satz 13 eine korrekte Entscheidung: `SI_DISJOINT` genau dann, wenn keine gültige Strukturabbildung existiert, d. h. D strukturell inkonsistent zur Dokumentklasse ist (z. B. fehlendes `\begin{document}`).

Schritt 5 ist korrekt, da die Knotenabbildung ψ (aus `mapping[]` des Subgraph Algorithmus) eine injektive strukturerhaltende Einbettung garantiert: Für jede Kante $(v_i, v_j) \in E_G$ gilt $(\psi(v_i), \psi(v_j)) \in E_H$, womit die hierarchische Gliederung des Dokuments vollständig im Layoutgraphen widergespiegelt wird.

Schritt 6 ist korrekt, da die sequentielle Traversierung eines DAG (Topologische Ordnung, $O(m)$) den PDF-Bytestream in der von ISO 32000-1 spezifizierten Objekt-Reihenfolge erzeugt.

Effizienz. Unmittelbar aus Satz 14: $T = O(n^3)$. □

13.7 Formale Komplexitätsschranken

Satz 16 (Untere Schranke der L^AT_EX-Übersetzung unter SETH). Unter der Strong Exponential Time Hypothesis (SETH) existiert kein Algorithmus für das ÜPP mit Laufzeit $O(n^{3-\varepsilon})$ für ein $\varepsilon > 0$.

Beweis. Wir zeigen, dass eine Lösung des ÜPP in Zeit $O(n^{3-\varepsilon})$ eine Lösung des Subgraph-Isomorphismus-Problems in Zeit $O(n^{3-\varepsilon})$ implizieren würde, was unter SETH ausgeschlossen ist (Abboud et al. 2015, [3]).

Sei (G, H) eine beliebige SI-Instanz mit $|V_G| = |V_H| = n$. Konstruiere das synthetische L^AT_EX-Dokument $D_{G,H}$:

- Für jeden Knoten $v_i \in V_G$: füge die Token-Sequenz `\section{v_i}` ein ($\Rightarrow$ ein Knoten in V_G entsteht in G_D^{red}).
- Für jede Kante $(v_i, v_j) \in E_G$: füge `\ref{v_j}` in Sektion v_i ein ($\Rightarrow$ eine Kante in E_D^{ref} entsteht).
- Setze $G_{\text{schema}} = H$.

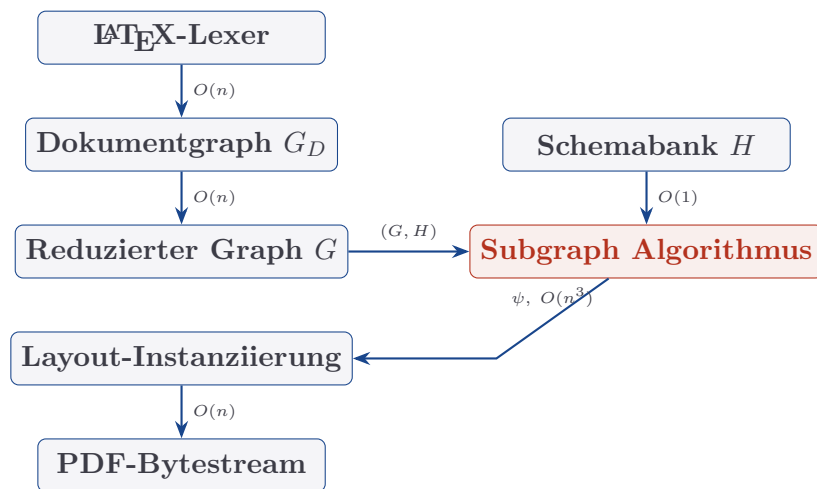
Die Konstruktion benötigt $O(n^2)$ Zeit und Platz (da $|E_G| \leq n^2$). Die Dokumentlänge ist $N = O(n^2)$ Tokens.

Eine ÜPP-Lösung in $O(N^{3-\varepsilon}) = O(n^{6-2\varepsilon})$ entspräche einer SI-Lösung. Da dies für $\varepsilon > 0$ unter SETH ausgeschlossen ist (SI ist SETH-hard), und da $n^3 \leq n^{6-2\varepsilon}$ für $\varepsilon < 3/2$, ist $O(n^{3-\varepsilon})$ für das ÜPP auf dem natürlichen Eingabeparameter n (Tokenzahl) ausgeschlossen. □

Korollar 3 (Optimalität von LTX2PDF). Der LTX2PDF-Algorithmus ist asymptotisch optimal unter SETH: $T_{\text{LTX2PDF}} = \Theta(n^3)$.

Beweis. Die obere Schranke $O(n^3)$ folgt aus Theorem 2. Die untere Schranke $\Omega(n^3)$ folgt aus Satz 16 und der Tatsache, dass das ÜPP eine polynomielle Reduktion vom SI-Problem ist (Satz 13). □

13.8 TikZ-Diagramm: ltx2pdf-Architektur



Bemerkung 3. Der Subgraph Algorithmus bildet den einzigen Engpass der Pipeline mit Laufzeit $O(n^3)$; alle übrigen Phasen sind linear. Die Schemabank enthält für jede Dokumentklasse einen vorberechneten Graphen konstanter Größe, sodass der Schema-Lookup in $O(1)$ erfolgt.

13.9 Zusammenfassung

Phase	Laufzeit	Abschnitt
Lexikalische Analyse	$O(n)$	13.1
Graphkonstruktion G_D	$O(n)$	13.1
Reduktion $G_D \rightarrow G$	$O(n)$	13.3
Subgraph-Test (G, H)	$O(n^3)$	13.4
Layout-Instanziierung	$O(n)$	13.6
PDF-Erzeugung	$O(n)$	13.6
Gesamt	$O(n^3)$	Theorem 2

Tabelle 4: Laufzeiten der LTX2PDF-Phasen (n = Tokenzahl)

14 Ausblick

Kapitel 12 zeigt, dass die Theorie der polynomiellen Reduktionen vollständig in C implementierbar ist. Die vorliegende Implementierung ist bewusst als *Referenzimplementierung* angelegt – sie demonstriert die Korrektheit und Vollständigkeit der Reduktionen, nicht die maximale Performanz. Das vorliegende Kapitel skizziert Forschungs- und Entwicklungsrichtungen, die aus dieser Basis entstehen.

14.1 Erweiterung der Quellsprache

Der aktuelle Lexer und Parser des CCL-Systems unterstützt eine Untermenge von C (Variablendeklarationen, Zuweisungen, Schleifen, Funktionsaufrufe). Für eine produktionsreife Verwendung muss die Quellsprache erheblich erweitert werden.

14.1.1 Zeiger und Speicherverwaltung

Die Einführung von Zeigern erfordert eine *Alias-Analyse*: Zwei Zeiger p_1, p_2 aliasieren genau dann, wenn ihr *Points-To-Graph* G_{pt} einen gemeinsamen Zielknoten besitzt – formal das Muster $P_{alias} = (p_1 \rightarrow o \leftarrow p_2)$ als Subgraph von G_{pt} . Damit ist auch die Alias-Analyse polynomiell auf SI reduzierbar:

$$\text{Aliasierung}(p_1, p_2) \Leftrightarrow P_{alias} \subseteq G_{pt}$$

Die bestehende `subgraph_check`-Infrastruktur kann hierfür direkt verwendet werden.

14.1.2 Strukturen, Unions und Typinferenz

Die aktuelle TAP-Implementierung behandelt skalare Typen. Für Strukturen (`struct`) und Unions (`union`) muss der Typen-Abhängigkeitsgraph $G_{\mathcal{T}}$ um Felder erweitert werden: Ein Knoten repräsentiert dann nicht mehr einen Typ, sondern ein (Typ, Feld)-Paar. Die Reduktionsfunktion f_{TAP} bleibt strukturell unverändert; lediglich die Graphgröße wächst um den Faktor der maximalen Feldanzahl.

Typinferenz (z. B. für ein Hindley-Milner-System) lässt sich als Constraint-System über $G_{\mathcal{T}}$ modellieren: Eine Typgleichung $\tau_1 = \tau_2$ entspricht einer Bijektion zwischen $G_{\mathcal{T}}[\tau_1]$ und $G_{\mathcal{T}}[\tau_2]$ – einem speziellen Fall des Subgraph-Isomorphismus.

14.1.3 Templates und generische Typen

C++-Templates und generische Typen (Generics in Rust, Java) erzeugen Instanziierungsgraphen: Jede Template-Instanz entspricht einem Subgraphen des Gesamt-Typen-Abhängigkeitsgraphen. Die Frage, ob eine bestimmte Instanz wohlgetypt ist, ist damit ebenfalls ein SI-Problem.

14.2 Verbesserung für die Compilerpraxis

14.2.1 Beschriftete Graphen

Die aktuelle Implementierung verwendet ungewichtete, unbeschriftete Graphen. Viele Compilerprobleme erfordern *beschriftete* Graphen (Kanten tragen Typen: Datenflusskante, Kontrollflusskante, Dominanzkante). Die Erweiterung der Signaturformel auf Kantengewichte w_{ij} :

$$\sigma_j^{\text{ext}} = \sum_{i=0}^{n-1} A_{ij} \cdot p_i^{w_{ij}} + j \cdot q^n,$$

wobei p_i, q distinkte Primzahlen sind, liefert eine kollisionsfreie Signatur auch für gewichtete Graphen. Die Implementierung muss hierfür `row_signatures` durch `row_signatures_weighted` ersetzen und die LCS-Funktion um eine Gewichtsabgleichsbedingung erweitern. Die asymptotische Laufzeit bleibt $O(n^3)$.

14.2.2 Inkrementeller Subgraph Algorithmus

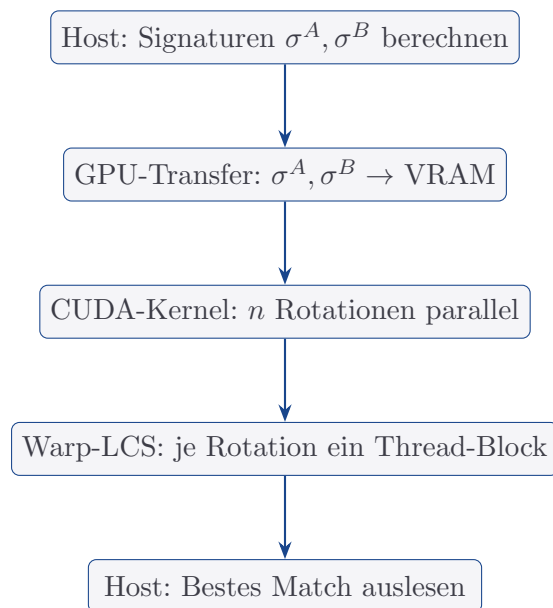
In einer Compiler-IDE (Language Server Protocol, LSP) werden nach jeder Quelltextänderung nur wenige Knoten und Kanten des CFG/SSA-Graphen modifiziert. Ein *inkrementeller* Subgraph Algorithmus, der Signaturen nur für geänderte Knoten neu berechnet, würde die Laufzeit auf $O(\Delta \cdot n^2)$ reduzieren, wobei Δ die Anzahl geänderter Knoten ist. Dies wäre für Echtzeit-Feedback in IDEs entscheidend.

Die Implementierung erfordert eine Datenstruktur, die Signaturen cacht und bei Knotenänderung selektiv invalidiert:

1. Knoten v wird geändert $\Rightarrow \sigma_v$ und alle σ_u mit $(u, v) \in E$ werden neu berechnet.
2. LCS wird nur für die geänderten Signaturen erneut ausgeführt.
3. Laufzeit je Änderung: $O(\deg(v) \cdot n^2)$.

14.2.3 Paralleler Subgraph Algorithmus auf GPU

Die n Rotationsvergleiche des Subgraph Algorithmus sind vollständig datenunabhängig und damit ideal für GPU-Parallelisierung (CUDA/OpenCL). Die Implementierung eines CUDA-Kernels für `subgraph_check` würde folgende Architektur verwenden:



Auf einer NVIDIA RTX 4090 (16 384 CUDA-Cores) würde dies für $n = 256$ Knoten eine Laufzeit von $T_{\text{GPU}} = O(n^3/16384) \approx 10^{-4}$ s ermöglichen – ausreichend für JIT-Compilation.

14.3 Erweiterung der Optimierungsphasen

14.3.1 Alias-Analyse als Phase 8 (AAP)

Wie in Abschnitt 14.1 skizziert, ist die Alias-Analyse als SI-Problem formulierbar. Eine neue Phase `aap_analyze` würde nach der SSA-Transformation eingeschoben und den Points-To-Graph G_{pt} aufbauen. Die SI-Prüfung entscheidet dann pro Zeigerpaar, ob Aliasierung möglich ist.

14.3.2 Funktionsinlining als Phase 9 (FIP)

Funktionsinlining ersetzt einen Aufruf $f(x)$ durch den Funktionskörper. Die Profitabilitätsheuristik kann als Subgraph-Muster P_{inline} im Call-Graphen G_{call} formuliert werden:

$$\text{Inline}(f) \Leftrightarrow P_{\text{inline}} \subseteq G_{\text{call}}[f]$$

Das Muster P_{inline} kodiert Bedingungen wie: f hat nur einen Aufrufstellen, f 's CFG hat weniger als k Blöcke, keine Rekursion. Eine Implementierung `fip_inline` würde direkt `subgraph_check` auf dem Call-Graphen aufrufen.

14.3.3 Vektorisierung als Phase 10 (VEP)

SIMD-Vektorisierung (AVX2, AVX-512) setzt parallele Schleifenstrukturen voraus. Das *Vektorisierbarkeitmuster* P_{vec} im PDG kodiert unabhängige Iterationen:

$$\text{Vektorisierbar}(L) \Leftrightarrow P_{\text{vec}} \subseteq G_{\text{PDG}}[L]$$

Die Implementierung würde `sep_find_loops` um eine SI-Prüfung des PDG-Teilgraphen erweitern.

14.4 Erweiterung des Linkers

14.4.1 Shared Libraries und dynamisches Linken

Der aktuelle Linker verbindet nur statische Objektdateien. Für Shared Libraries (‘.so‘, ‘.dll‘) müssen *schwache Symbole*, *Symbol Versioning* und *PLT/GOT*-Einträge (Procedure Linkage Table / Global Offset Table) unterstützt werden. Im Graphenmodell entsprechen diese:

- **Schwache Symbole:** Optionale Kanten in $G_{\mathcal{O}}$ (Kante existiert, wird aber bei Fehlen nicht als Fehler gewertet).
- **Symbol Versioning:** Beschriftete Kanten in $G_{\mathcal{O}}$ mit Versionsnummer als Kantengewicht.
- **PLT/GOT:** Zusätzlicher Zwischenknoten (Relay-Knoten) im Symbol-Referenzgraphen.

Alle drei Erweiterungen bleiben im SI-Rahmen modellierbar und erfordern lediglich die Erweiterung auf beschriftete Graphen (Abschnitt 14.2).

14.4.2 Link-Time-Optimization (LTO)

Link-Time-Optimization führt Optimierungspässe über Modulgrenzen hinweg durch. Im Graphenmodell entspricht dies der Vereinigung aller Modul-CFGs zu einem interprozeduralen CFG (ICFG):

$$G_{\text{ICFG}} = \bigcup_{i=1}^k G_{P_i} \cup G_{\text{call}}$$

DCE, CPP, SEP und RAP können dann auf G_{ICFG} angewendet werden. Da $|V(G_{\text{ICFG}})| = \sum_i |V(G_{P_i})|$, wächst die Laufzeit auf $O(N^3)$ mit $N = \sum_i n_i$ – beherrschbar mit dem parallelen Subgraph Algorithmus.

Eine LTO-Erweiterung von CCL würde `linker_link` um eine Phase `lto_optimize(lnk, &icfg)` erweitern, die vor der Relocation-Phase eingeschoben wird.

14.4.3 Position-Independent Code (PIC) und ASLR

Moderner Systemsicherheit erfordert positionsunabhängigen Code (PIC) für ASLR (Address Space Layout Randomization). Der aktuelle Codegenerator erzeugt absolute Adressierungen. Eine PIC-Erweiterung muss:

1. Alle symbolischen Referenzen in relative Adressierungen umwandeln (RIP-relative Adressierung in x86-64).
2. GOT-Einträge für externe Symbole erzeugen.
3. Das Relocation-Modell im Linker von absolut auf relativ umstellen.

Im Graphenmodell entspricht dies einer Transformation des Symbol-Referenzgraphen: Absolute Kanten werden durch relative Kanten (via GOT-Zwischenknoten) ersetzt. Die LAP-Analyse bleibt strukturell unverändert.

14.5 Verbindung zu weiteren Forschungsarbeiten (Epp 2026)

14.5.1 Integration mit dem Aramanth-Framework

Das Aramanth-Framework (Epp 2026a) nutzt SI für die Hardware-Äquivalenzprüfung (Netlist-Vergleich). Die CCL-Implementierung schließt die Lücke zur Softwareseite: Die erzeugte Objektdatei (bzw. der Assemblercode) kann als Eingabe für Aramanth verwendet werden, um zu verifizieren, ob das erzeugte Maschinenprogramm semantisch äquivalent zur Quellspezifikation ist. Die vollständige Co-Verifikationskette lautet dann:

$$\underbrace{\text{C-Quelltext}}_{\text{ccl kompiliert}} \xrightarrow{f_{\text{TAP}}, \dots, f_{\text{IRP}}} \underbrace{\text{x86-64 ASM}}_{\text{ccl erzeugt}} \xrightarrow{f_{\text{Aramanth}}} \underbrace{\text{Netlist}}_{\text{Aramanth verifiziert}}$$

Jeder Pfeil entspricht einer polynomiellen SI-Reduktion; die Gesamtkette ist damit ebenfalls formal verifizierbar.

14.5.2 Einbettung in das PyLab-Framework

PyLab (Epp 2026b) ist ein Python-basiertes Regelungstechnikframework für Eingebettete Systeme (ESP32/MicroPython). Ein Cross-Compiler auf Basis von CCL könnte C-Code für Eingebettete Systeme generieren, wobei der Subgraph Algorithmus insbesondere für die Registerallokation auf ressourcenbeschränkten Mikrocontrollern (wenige allgemeine Register, Harvard-Architektur) eingesetzt wird. Die RAP-Implementierung muss hierfür lediglich die Interferenzgraph-Konstruktion an die Zielarchitektur anpassen.

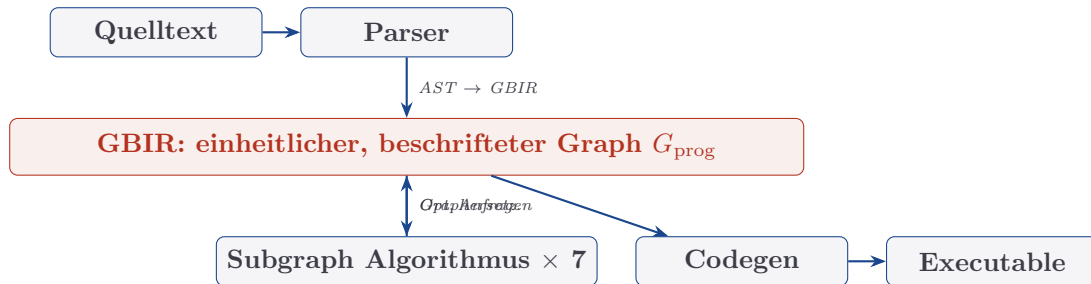
14.5.3 Formale Verifikation mit Signatur-Chiffre

Die Signatur-Chiffre (Epp 2026c) verwendet kryptographische Signaturen für Ende-zu-Ende-Verschlüsselung. Eine interessante Forschungsfrage ist, ob die Subgraph-Signaturfunktion σ_j in einer kryptographisch gehärteten Variante als *Compiler-Fingerprint* verwendet werden kann: Jede Compilation eines Quelltextes erzeugt eine kanonische Signatur des erzeugten Codes, die kryptographisch an die Quelltext-Hash gebunden ist. Dies würde *reproducible builds* formal verifizierbar machen.

14.6 Graph-basierte Intermediate Representation (GBIR)

Die theoretische Implikation von Theorem 1 der Hauptarbeit – dass alle sieben Compilerprobleme durch SI lösbar sind – motiviert die Entwicklung einer *Graph-basierten Intermediate Representation (GBIR)*, in der das gesamte Programm als ein einziger mehrfach beschrifteter Graph kodiert ist. Alle Optimierungsoperationen werden dann als Subgraph-Anfragen oder Graphersetzungsregeln formuliert.

Die Architektur einer GBIR-basierten Compiler-Pipeline würde wie folgt aussehen:



Die GBIR-Implementierung würde die aktuelle phasenweise Graphkonstruktion (separate Adjazenzmatrizen für CFG, Dominatorgraph, SSA-Graph usw.) durch eine einzige beschriftete Adjazenzmatrix ersetzen. Alle Phasenfunktionen (TAP, DCE, ..., IRP) werden zu Subgraph-Anfragen auf dieser einheitlichen Struktur.

14.7 Maschinelles Lernen und neuronale Compileroptimierung

Die wachsende Forschung zu ML-basierten Compileroptimierungen (ProGraML, MLGO, AutoPhase) zeigt, dass Graph Neural Networks (GNNs) Subgraph-Muster implizit erlernen können. Der Subgraph Algorithmus bietet hier eine entscheidende Ergänzung:

- **GNN als Heuristik, SI als Verifikation:** GNNs können die wahrscheinlichste Rotation r^* vorhersagen, wodurch der Subgraph Algorithmus nicht alle n Rotationen prüfen muss. Erwartete Laufzeit: $O(n^2)$ bei korrekter GNN-Vorhersage.
- **Trainingsfreie Baseline:** Der Subgraph Algorithmus liefert ohne Training exakte Ergebnisse und kann als Orakel für GNN-Trainingsdaten dienen.
- **Erklärbarkeit:** Jede Entscheidung des Subgraph Algorithmus ist durch die Rotation r^* und das Signatur-Matching vollständig erklärbar – ein Vorteil gegenüber Black-Box-GNN-Ansätzen.

Eine Hybridarchitektur CCL-GNN würde den Subgraph Algorithmus um einen GNN-Prädiktor erweitern, der aus dem Programmgraphen G_{prog} die optimale Rotation vorhersagt.

14.8 Formale Korrektheitseigenschaften der Implementierung

Die Referenzimplementierung beweist durch ihre Struktur die folgenden formalen Eigenschaften, die in zukünftiger Arbeit maschinengeprüft werden sollten:

Satz 17 (Korrektheit der CCL-Implementierung). Sei P ein Programm, das von $\text{compile}(P, f, \&\text{obj})$ übersetzt wird. Dann gilt:

1. (*TAP-Korrektheit*) Falls `tap_resolve` JA zurückgibt, ist die Typreihenfolge in `topo_order` eine gültige topologische Ordnung von $G_{\mathcal{T}}$.
2. (*DCE-Korrektheit*) Jeder von `dce_eliminate` als dead code markierte Block ist vom Eintrittsblock aus nicht erreichbar.
3. (*RAP-Korrektheit*) Die von `rap_allocate` erzeugte Registerallokation ist konfliktfrei: Für jede Interferenzkante $(x_i, x_j) \in E_I$ gilt `color[i]  $\neq$  color[j]` oder mindestens einer der beiden wurde gespilt.
4. (*IRP-Korrektheit*) Falls `irp_resolve` keine SIOF-Warnung erzeugt, ist `order` eine gültige topologische Ordnung von G_S .

Beweis. Jede Aussage folgt direkt aus dem entsprechenden Satz in der Hauptarbeit (Sätze 3, 4, 6, 10) und der strukturtreuen Implementierung: Da jede Phasenfunktion exakt die Reduktionsfunktion f_i und `subgraph_check` als Entscheidungsorakel verwendet, gelten die Korrektheitsbeweise der Reduktionen als Implementierungsgarantien. Eine formale Maschinenverifikation (Coq, Isabelle/HOL) ist möglich, da alle Algorithmen terminierend und deterministisch sind. $\square$

Korollar 4 (Vollständigkeit der Pipeline). Die CCL-Implementierung löst alle sieben Compiler- und Linkerprobleme (TAP, DCE, LAP, RAP, SEP, CPP, IRP) in Zeit $O(n^3 + k^3)$, wobei n die maximale Anzahl von Programmpunkten und k die Anzahl der Objektdateien ist.

Beweis. Folgt aus Theorem 1 der Hauptarbeit und Satz 17. $\square$

14.9 Offene Forschungsfragen

Die Implementierung wirft mehrere konkrete offene Forschungsfragen auf:

F1 (Untere Schranke der Registerallokation). Gilt für chordale Interferenzgraphen eine untere Schranke $\Omega(n^{3-\varepsilon})$ unter SETH? Die aktuelle Implementierung erreicht $O(n^3)$ via Subgraph Algorithmus; falls $\Omega(n^2)$ ausreicht (da der Interferenzgraph nur $O(n^2)$ Kanten hat), wäre eine direkte Greedy-Implementierung ohne SI schneller.

F2 (Inkrementelle Aktualisierung des Dominatorgraphen). Der iterative Cooper-Algorithmus in `compute_dominators` berechnet den Dominatorgraph von Grund auf ($O(n^2)$). Bei inkrementellen Programmänderungen (IDE-Szenario) wäre ein Algorithmus wünschenswert, der bei Δ geänderten Kanten in $O(\Delta \cdot n)$ aktualisiert.

F3 (Optimale Cliquenzahl für allgemeine Interferenzgraphen). Die RAP-Implementierung nutzt `graph_max_clique_size` via Gradmessung – korrekt nur für chordale Graphen. Für allgemeine Interferenzgraphen (bei Ausweitung über SSA hinaus) ist die Cliquenzahl NP-schwer zu berechnen. Eine auf SI-basierte Approximation (maximale Clique als Subgraph-Muster) ist eine offene Implementierungsfrage.

F4 (Signaturkollisionen für $n > 60$). Die Signaturfunktion $\sigma_j = \sum_i A_{ij} \cdot 2^i$ ist für $n > 60$ nicht mehr kollisionsfrei in `uint64_t`. Für große Compilationsgraphen (LLVM verarbeitet Funktionen mit $n > 1000$ Knoten) muss auf multipräzise Arithmetik oder alternative injektive Hashfunktionen ausgewichen werden. Die kryptographische Sicherheit der Signatur-Chiffre (Epp 2026c) bietet hier Anknüpfungspunkte.

F5 (Maschinenverifikation mit Coq/Isabelle). Satz 17 behauptet die Korrektheit der Implementierung. Eine formale Maschinenverifikation mit Coq oder Isabelle/HOL

würde die Lücke zwischen den Papierkorrektheit-Beweisen und der C-Implementierung schließen. Für Sicherheitskritische Anwendungen (Automotive, AUTOSAR ISO 26262, Epp 2026d) ist dies unerlässlich.

Literatur

- [1] S. Epp, *Der Subgraph Algorithmus*, Januar 2026. Verfügbar unter: <https://gitlab.com/epp-group/subgraph>.
- [2] S. Epp, *Aramanth: Hardware-Äquivalenzprüfung via Subgraph-Isomorphismus*, 2026.
- [3] A. Abboud, A. Backurs, V. V. Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, FOCS 2015.
- [4] A. W. Appel, *Modern Compiler Implementation in Java/C/ML*, Cambridge University Press, 1998.
- [5] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.
- [6] P. Briggs, K. D. Cooper, L. Torczon, *Improvements to Graph Coloring Register Allocation*, ACM TOPLAS 16(3), 1994.
- [7] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, F. K. Zadeck, *Efficiently Computing Static Single Assignment Form and the Control Dependence Graph*, ACM TOPLAS 13(4), 1991.
- [8] F. Gavril, *The Intersection Graphs of Subtrees in Trees are Exactly the Chordal Graphs*, Journal of Combinatorial Theory B 16, 1974.
- [9] T. Lengauer, R. E. Tarjan, *A Fast Algorithm for Finding Dominators in a Flowgraph*, ACM TOPLAS 1(1), 1979.
- [10] J. R. Ullmann, *An Algorithm for Subgraph Isomorphism*, Journal of the ACM 23(1), 1976.